08/07/25

Program Vs Software

Program	Software
→ Program is a set of instruct" to perform a specific task	→ collection of program to execute whole task.
→ Program can be written by a single user.	→ It can be written by multiple users.
→ Program can't be a s/w.	→ s/w can be a program.
→ There is no concept of classification. eg: malware.	→ ~~divided~~ classified to system & application s/w eg: windows, adobe reader

Software Engineering

→ It is the product of two parts : one is software which can be defined as the collection of integrated program with documentation and operating procedures.

→ Engineering defines application of scientific and practical knowledge to invent, design, built, maintain and process etc.

→ Hence, software engineering is the branch related to evolution of software products using well-defined scientific principles techniques and procedures.

Importance of software Engineering

→ To manage large software
→ Cost management
→ more scalability (no. of program ↑, performace should'nt ↓)
→ To manage dynamic nature of software
→ Better quality management.
→ Reduce complexity.

Features of Good software Engineer

→ Good technical or domain knowledge
→ Good programming ability.
→ Good communicational skill
→ Intelligent and discipline.
→ He or she expose to systematic method that means familiar to engineering principles.

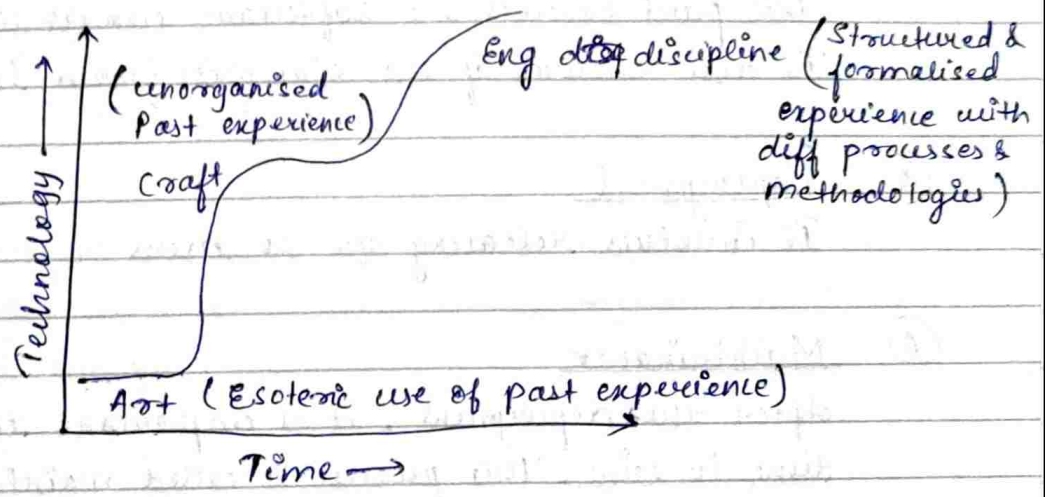
Evolution of s/w Engineering discipline

→ In 1960s 70s, the field of s/w engineering began to take shape.
→ S/w engineering principles have evolved over past 50 years with contribution from numerous researchers and s/w professionals.

→ The early programmers used an exploratory programming style (every programmer himself uses his own s/w development techniques guided by his own intuitions and experiences).

→ Over 50 years, software engineering has evolved from an art to craft then engineering discipline.
→ This transition (art → craft → engineering discipline) involved more formalised and structured approach.

→ The processes and methodologies were used to ensure the quality and reliability of S/W system.
→ The following diagram shows the evolution of S/W engineering.



10/07/25

Software Development Project

→ It involves the systematic process of creating and maintaining software applications.
→ This project typically follow a structured approach to develop an efficient software.

Key aspects :-

① Requirement Analysis

It includes gathering the requirements, understanding and documenting what the users need from a software.

② **Design (Blue print)**
This is the planning of architecture & components of the software.

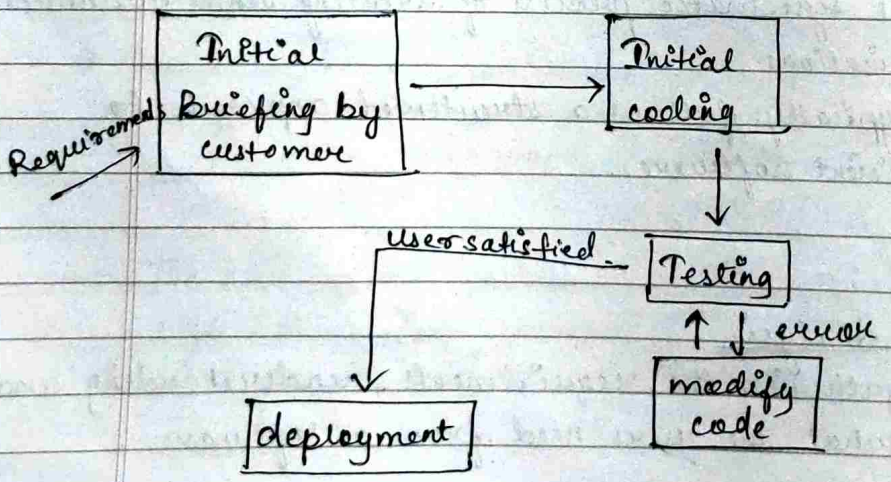
③ **Implementation**
This is the actual developing software that includes writing the actual code for software.

④ **Testing**
This part ensures the software works correctly or not. It also ensures if the s/w free from bugs or not.

⑤ **Deployment**
It involves releasing s/w to users

⑥ **Maintainance**
After the deployment, it is important to check the s/w time to time. This process is called maintainance that updates s/w and fix bugs over time.

Explanatory style of software development



→ It refers to informal development style or build and fix the style in which the programmer uses his own intuition to develop program.
→ Programmer skips the usage of systematic body of knowledge i.e, categorized under s/w engineering discipline.
→ This style of development gives complete freedom to programmers to choose activities to develop s/w.
→ This style doesn't offer any rules to start developing any s/w.

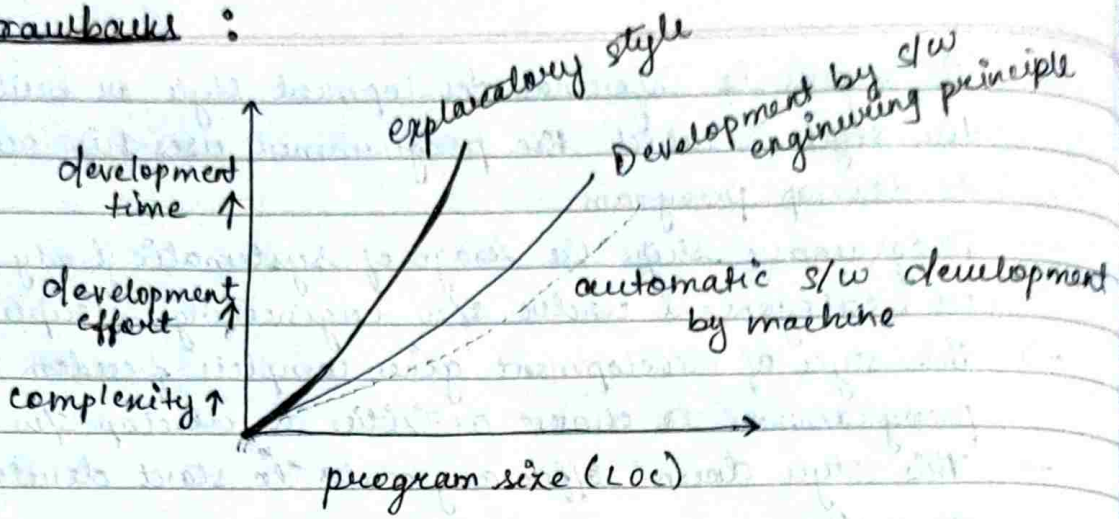
Initial Briefing
- Indicates brief introduction about s/w by customer.

Initial coding
- After the developer or programmer knew about the problem, he starts coding to develop a working program without any requirement analysis.

Test
- After above program will be tested i.e, to find or fix the bugs.
- This cycle continues until satisfactory code is not obtained.
- After finding satisfactory code development gets complete.

Usage
- Basically it use for development of small program use by students in labs or assignment.
- Not good for industry usage.

Drawbacks :



→ The dotted line used to represent a case when development is carried out by automated machine. In this, when increase in time, effort, size would linearly increased.

→ Thin solid line (Development by s/w engineering principles) It is also close to automated machine s/w ~~development~~ development approx increase the size almost linearly.

→ Thick solid line (exploratory) ⇒ as the program size increases, require effort and time increases exponential which cause difficulty in problem as it depends on human intuitions

14/07/25.

Emergence of software Engineering

The following are the innovation and programming experiences for development of software engineering discipline.

(i) Early computer programming (use of Assembly language)

→ Early commercial computers were very slow and lack of sophistication. Even a simple processing task took

high computational time even if the program size was very small.
→ Here, the programming style was exploratory style.

(ii) High level language programming
→ Computers became faster with introduction of semi-conductor technology.
→ It became possible to solve larger and more complex program using high level language like FORTRAN, OBDL etc
→ Here, efforts required to develop s/w was reduced but s/w developing style was still exploratory.

(iii) Control flow based design
→ As the size and complexity of program kept on increasing, the exploratory style proved to be insufficient as it was not only difficult to write program but also expensive.
→ So, program was gave attention to design programs control flow structure that indicates the sequence in which ~~prog~~ program's instructions are executed.
→ The control flow structure (flow chart) was developed but the main disadvantage was a program with complex flow chart is difficult to understand and ~~maintain~~ maintain.

(iv) Data structure oriented design
→ As computers became more powerful with advancement of integrated circuits (ICs) which were used to solve more complex problems.
→ Softwares engineers were then expected to develop larger and more complicated s/w products that required ten of thousand line of code.

→ The control flow structure could not satisfactory handle these problems.
→ So, the developers payed attention to design important data structure.
→ Designing techniques based on control structure principles is called data structure oriented design.
→ Using these techniques a program's data structure are designed then program design is derive from it's datastructure design.
eg → JSP (Jackson Structure programming in 1975 by Micheal Jackson).

(v) Data flow oriented design
→ With introduction of VLSI (Very Large Scale Integration) some new architectural concept, more complex and sophisticated s/w products were needed to solve then the data flow oriented design was introduced.
→ These techniques advocate major data items handle by system must be identified first and the processing required for data items are described secondly.
→ The data items are exchanged between different functions (processes) are represented in a diagram known as DFD (Data Flow Daigram).

(vi) Object Oriented Design (OOD)
→ The next development was Data flow oriented technique which was widely accepted because of it's simplicity, code and design and reuse concept.
With lower development, test and maintanance.
→ Here, the important objects in a problem are identified first then the relationship (abstract, inheritance) are determined.

24/07/25

Notable changes in Software development

There is a difference between an exploratory style of software development and another development effort based on ~~error~~ modern software ~~development approaches~~ engineering pratices.

The following noteworthy differences between these two software development approaches would be immediately observable :

i) → An important difference is that the exploratory software development style is based on error correction while the software engineering principles are based on error prevention.
→ Software engineering principles is the realisation that it is much more cost-effective to prevent errors from occuring than to correct them as and when they are detected.
→ In the exploratory style, errors are detected only during the final product testing.
→ In constrast, the modern practice of software development is to develop the software through several well-defined stages such as requirements specification, design, coding, testing etc and attempts are made to detect and fix as many errors as possible in the same phase in which they occur.

ii) → In exploratory style, coding was considered as synonyms with software development.
→ Exploratory programmers literally dive at the computer to get started with their programs even before they fully learn about the problem.
→ Exploratory programming was costly for non-trival problems as well as produces hard to maintain products.

→ In the modern software development style, coding is regarded as only a small part of overall software development activities.

→ There are several development activities such as design and testing which typically require much more effort than coding.

iii) → A lot of attention is paid to requirement specification.

→ Significant effort is devoted to develop a clear specification of the problem before any development activity is started.

iv) → There is a distinct design phase where standard design techniques are employed.

→ Periodic reviews are carried out during all stages of the development process. The

→ The main objective of carrying out reviews is phase containment of errors i.e, detect & correct errors as soon as possible.

→ Software testing has become very systematic & standard testing techniques are available.

v) → There is better visibility of design and code. Visibility means production of good quality, consistent & standard documents during every phase. In the past, very less attention was paid to producing good quality & consistent documents.

→ In exploratory style, the design & test activities even if carried were not documented satisfactorily. Today, consciously good quality documents are being developed during product development which has made the fault diagnosis & maintainance far more smoother.

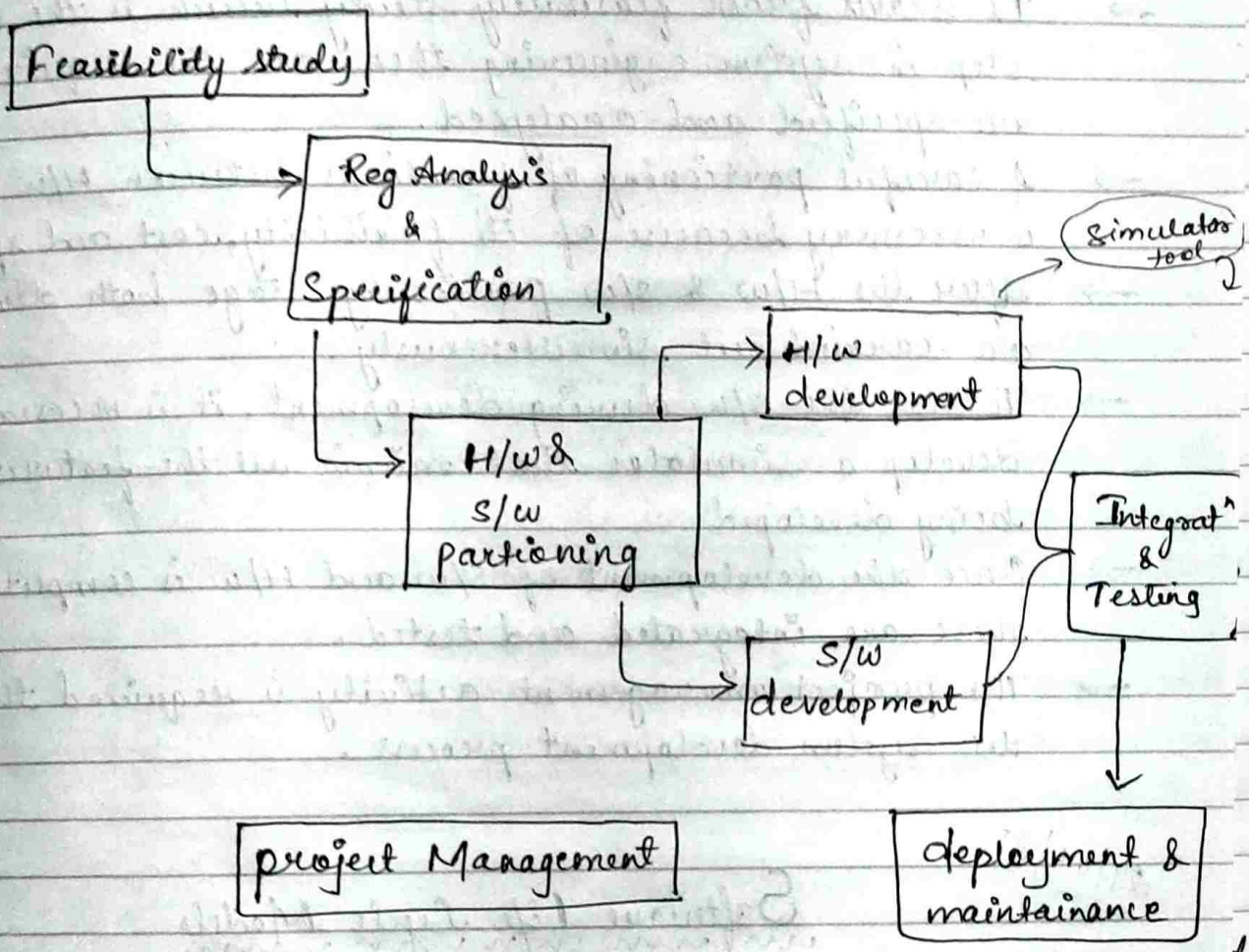
vi) → Now, projects are first thoroughly planned. Project planning normally includes preparation of various types of estimates, a resource scheduling & development of project tracking plans.

viii) Several metrics are used to help in s/w project management & s/w quality assurance.

25/07/25

~~Other developments~~

Computer System Engineering



→ I/e assume that the software to be developed would run on some general purpose hardware platform but some software is run only on some specific H/w.

→ Eg of this systems includes robot, factory automation system, mobile communication handset etc.

→ Hence, computer system engineering addresses the development of systems that require the development of s/w and specific H/w to run that software.

→ So, system engineering encompases s/w engineering.

diagram explanation

→ In the above diagram, shows the workflow of computer system engineering.
→ It starts from feasibility study which is the necessary step in system engineering then all the requirements are specified and analyzed.
→ A careful partioning of functions between H/w & S/w is necessary because of it's flexibility, cost and speed etc.
→ After the H/w & S/w partioning stage both developments are carried out simultaneously.
→ To test the S/w during development, it is necessary to develop a simulator that mimic all the features of H/w being developed.
→ Once the development of S/w and H/w is completed, then these are integrated and tested.
→ The project management activity is required throughout the system development process.

29/07/25

Software Life Cycle Models

→ A S/w lifecycle is the series of identifiable stages that a S/w product undergoes during it's lifetime.
→ The stages can be feasibility study, Requirement analysis specification, design, coding, testing and maintainance.
→ Each of these stages is called S/w lifecycle phase.
→ Hence, a S/w lifecycle model is a descriptive and diagramatic presentation of S/w lifecycle.
→ In otherwords, we can say, a lifecycle model maps different activities performed on a S/w product from it's inception to retirement.

There is a clear difference between methodology and process. A process covers all the activities beginning from product inception to delivery and retirement.
In contrast, a methodology covers only a single activity in it's individual ~~stage~~ state of development.

* Why to use lifecycle Model ?

→ The primary advantage is that it encourages development of software in a systematic and disciplined manner.
→ As we know, s/w product is developed by a team, so it is necessary to have a precise understanding among the team members as what to do and when to do. That leads to less project failure.
→ It also helps to solve the problem arising in each parts and smoothly manage overall development.

* Why it should be documented ?

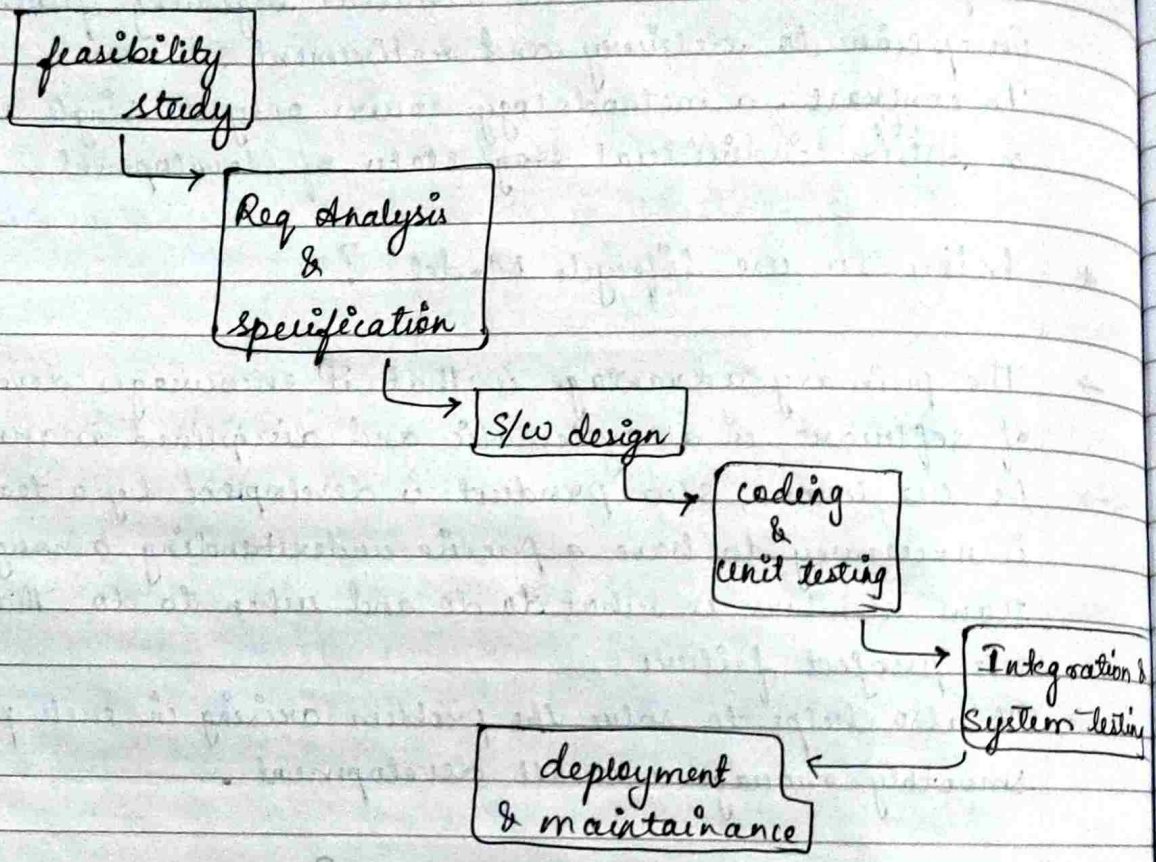
A documented lifecycle model prevents misinterpretation and helps to identify inconsistency, redundancy and errors in the development process.
It becomes easier to tailor a documented process model when required for specific project as it helps to identify all the requirements.

Phase
~~Page~~ entry and exit criteria

A phase can begin only when the phase entry criteria is satisfied and after completing all it's activities it goes to exit criteria.

31/07/25

Classical Waterfall Model



→ It is named as waterfall model because it's diagram cascade of waterfall.

→ The classical waterfall model is the most obvious way to develop software.

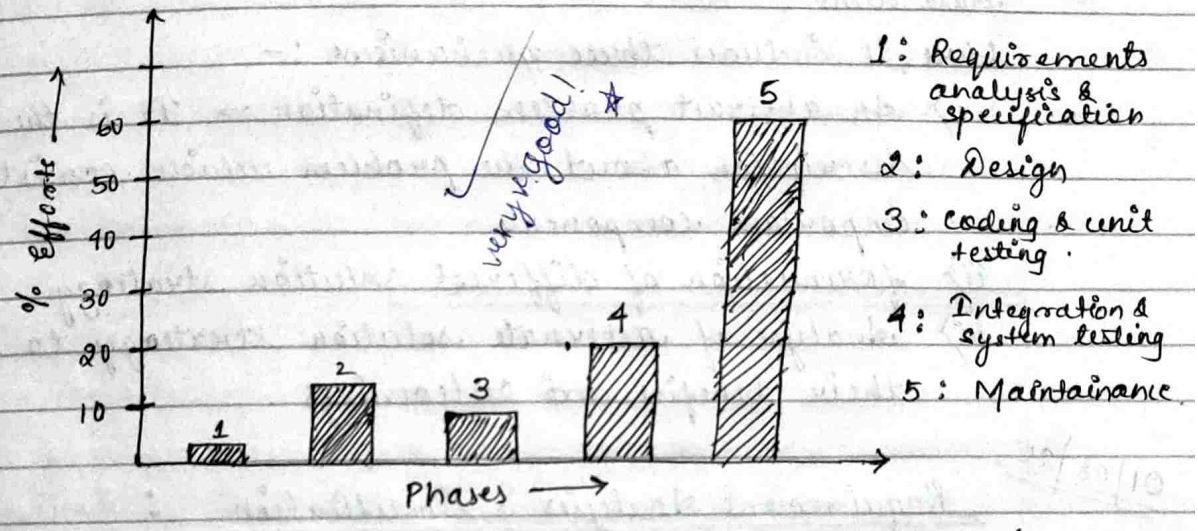
→ This model can be considered as theoritical way of developing software but it can't be used in actual development projects.

→ The above figure represents different phases of classical waterfall model that includes feasibility study, requirement analysis & specification, design, coding & unit testing, integration & system testing, deployment & maintainance.

→ From feasibility study to product delivery is known as development part.

→ The maintainance phase / part continues after the completion of development phase.

Efforts calculation in classical Waterfall Model



→ The above figure represents relative efforts distribution among different phases.

→ From the observation, we can conclude that among all the lifecycle phases, the maintainance phase normally requires the maximum efforts i.e, 60%. However among the development phase, the integration and system testing phase requires maximum efforts for development of typical product.

→ Each phase of the waterfall model should contain well-defined starting and ending criteria and it should be documented in the form of test ~~descriptions~~ description.

Feasibility Study :

↳ It is the basic and important step where it aims to determine whether it would be financially and technically feasible to develop the product.

↳ It involves the analysis of problem & collection of relevant information related to product and different data items.

Here, it includes three mechanisms :-

i) An abstract problem defination ⇒ It is the raw description about the problem which consist only important components.

ii) formulation of different solution strategy

iii) Analyse of alternate solution strategy to compare their benefits and outcomings.

01/08/25

Requirement Analysis & Specification :

↳ The aim of this phase is to understand the exact requirements of the customer and document them properly

↳ This phase consist of two parts :-

i) Requirement gathering & analysis

ii) Requirement specification

i) Requirement gathering & analysis –

- The goal of requirement gathering is to collect all the relevant information from the customers regarding the product with a clear understanding of customer requirements.
- These data are collected from customers through interviews and discussions.
- After collecting the requirements, the analysis step is started for weeding out incompleteness & inconsistency

in these requirements (* inconsistent requirement is one where some parts of the requirement contradicts with other part) (* incomplete requirement is one where some part of requirement may be omitted altogether.

ii) Requirement Specification –
After the requirement gathering & analysis activity, the identified requirements are organised into SRS documents (software Requirement Specification) which consist mainly three components such as functional requirement, non-functional requirement and goal of implementation.

Design :

↳ The goal of design phase to transform the requirements specified in SRS document into a structure which is suitable for implementation.
↳ There are two approaches :-
i) Traditional design approach –
This consist of two parts –
a) structure analysis b) Structure design
- In structure analysis, the detailed structure of problem is examined.
- This is followed by structure design activity.
- During structure design, the result of structure analysis is converted into software design.

ii) Object oriented design approach –
It is a new technique where various objects that occur in problem domain is first identified then their relationship that exist among objects are identified.

Coding & Unit Testing :

↳ The purpose of the stage is to translate the software design into source code.

↳ Each component of design is implemented as program module. The end product of this phase is a set of program modules that have been individually tested which is known as unit testing.

↳ This is the most obvious way to debug errors identified in this stage.

Integration & System Testing :

↳ After unit testing, this phase is carried out where modules are integrated in a planned manner that means incremental over one steps, then the integration testing carried out.

↳ After the successful integration testing, system testing is carried out to ensure that the developed system confirm it's all requirements mentioned in SRS document.

↳ It consist of three activities :-

i) α-testing -

- It is performed by development team itself.

ii) β-testing -

- It is performed by friendly customers

iii) acceptance testing -

- It is performed by real time customers

- After the product delivery whether to accept or reject the product.

Maintainance :

It is the last phase which requires much more efforts than the development phase.
If we compare these two phases it can be represented as 40:60 ratio where 40% is development phase and 60% is maintainance phase.
It also consist three activities :-

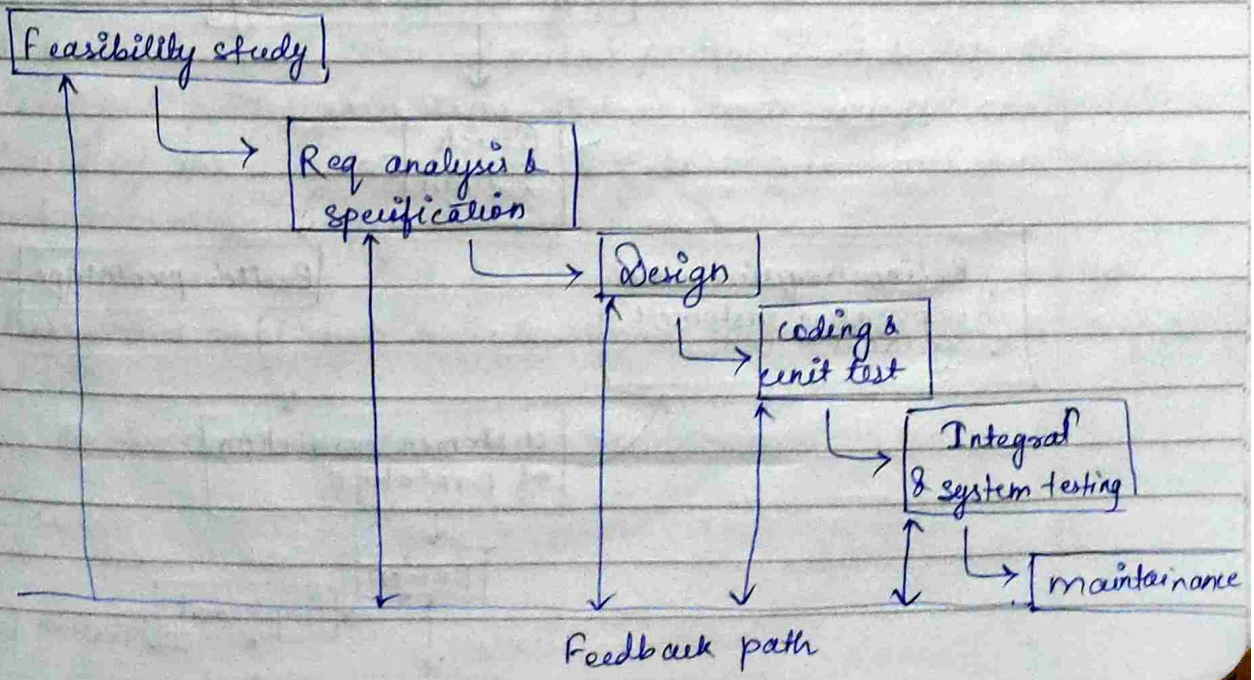
i) corrective -
It involves correcting errors that are not discovered during product development phase.

ii) perfective -
It involves Improving the functionalities of the system.

iii) adaptive -
Porting the new software into new environment to check it's functionality.

05/08/25

Iterative Waterfall Model



↳ The classical waterfall model is good for theoritical approach, even in practical development environments the engineers do commit a large no. of errors in almost every phase of the life cycle.

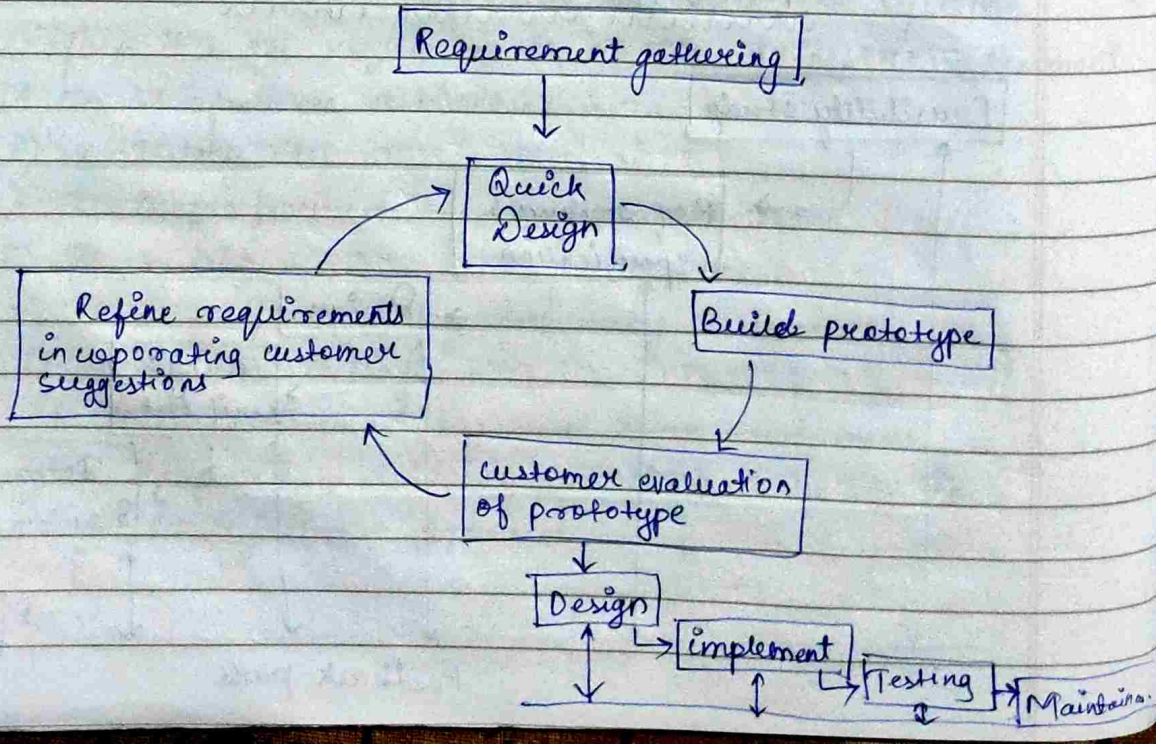
↳ The source of defects can be oversight assumption use of inappropriate technology & communication gaps.

↳ Once, a defect is detected, the engineers need to go back to that specific phase to redo the work again which is not possible in classical waterfall model.

↳ In practical development, a feedback path is needed in classical waterfall model for correction of errors and redoing the work, which is nothing but the iterative waterfall model.

Internal

Prototype Model



↳ Prototype type is the preliminary version of actual software which is carrying out before the development of actual software system.

↳ A prototype is a toy implementation of system which exhibits limited functional capabilities, low reliability and performance compared to actual software. Hence, prototype is known as crude (rough) version of software which is used when the technical solutions are unclear to development team. So, a developed prototype can help engineers to critically examine the technical issues associated with product development.

↳ The another reason of using this model because it is impossible to get the right software for the first time. So, this model can be used to develop a good product later which is advocated by Brooks in 1975.

↳ In the above diagram, the product development starts with initial requirement gathering phase.
↳ A quick design is carried out & prototype is built, the developed prototype is submitted to customers for evaluation.
↳ Based of the customer feedback, the requirements are refined and prototype is again modified.
↳ This cycle continues till the customer approves the prototype.
↳ Then the actual system is developed using iterative waterfall model.

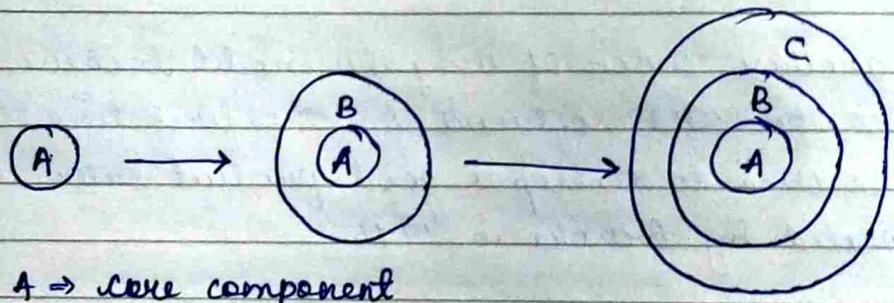
07/08/25

Evolutionary Model

↳ The lifecycle of this model is referred as successive version model and sometimes called as incremental model

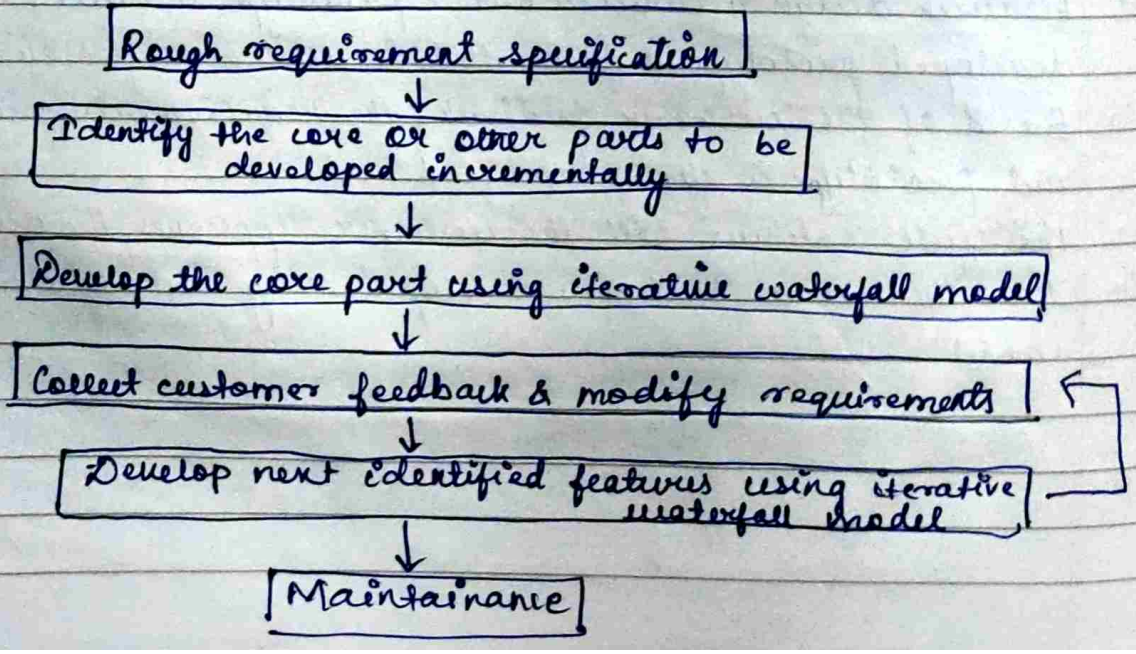
↳ In this, the software is first broken down into several modules which are incrementally constructed and delivered

↳ The development team first develops the core module which is refined into increasing levels of capabilities, by adding new functionalities in successive version this model also follows iterative waterfall model.



A ⇒ core component

Evolutionary Model of Software development



In evolutionary model, each successive model is fully functioning software, performing more useful work than the previous versions.

Advantages :

↳ Users get chance to experiment with partially developed software, for which this model helps to accurately collect users requirement.

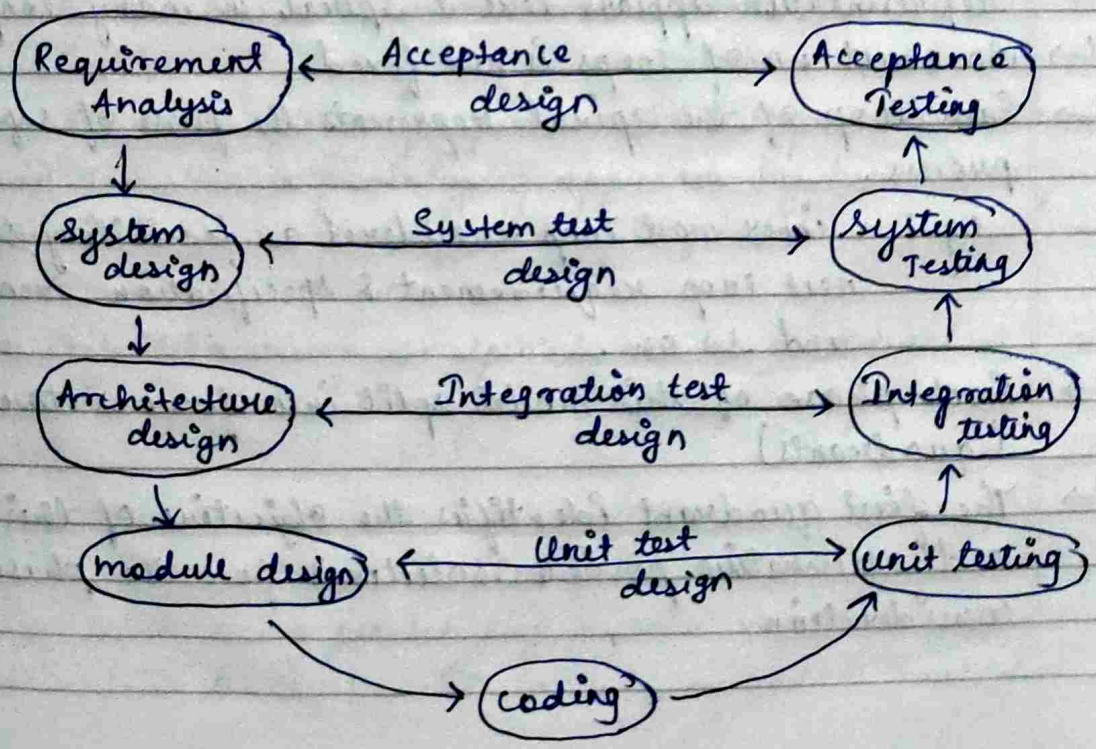
↳ The product is tested thoroughly with reduced no of errors

Disadvantages :

↳ It requires more resources

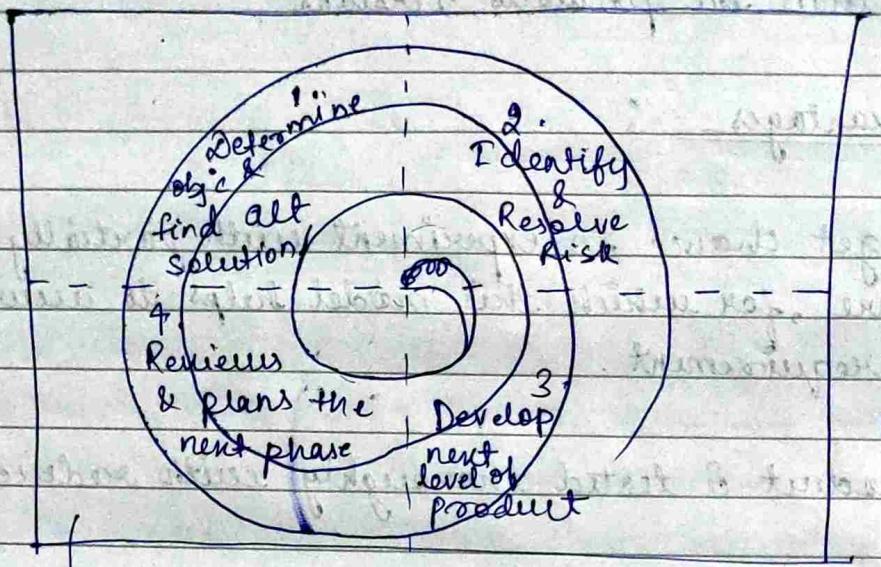
↳ Sometimes it is very difficult to implement it in real life.

V - Model



12/08/25

Spiral Model



(Boehm 1986)

(Meta Model)

→ waterfall + prototype + Evolutionary

↳ It is developed by Boehm in 1986.

↳ It is known as spiral model because it's diagramatic representation appears like a spiral in many loops.

↳ An exact no. of loops is not fixed.

↳ Each loop of the spiral represents the phase of software process.

eg :- inner most loop considered as feasibility study next loop requirement & specification then design and so on.

↳ Each phase of this model split into four sectors (quadrants).

↳ The first quadrant identifies the objective of this phase and alternative possible solutions for the phase under consideration.

↳ Second quadrant evaluates alternative solutions to select based solutions in which the potential risk are identified and deal with developing an appropriate prototype.

↳ Risk is an essential adverse circumstances that hampers the successful completion of software project.

↳ The third quadrant consist of developing and verifying next level of product.

↳ Activities during fourth quadrant concern reviewing the result with customer and planning next iteration around spiral.

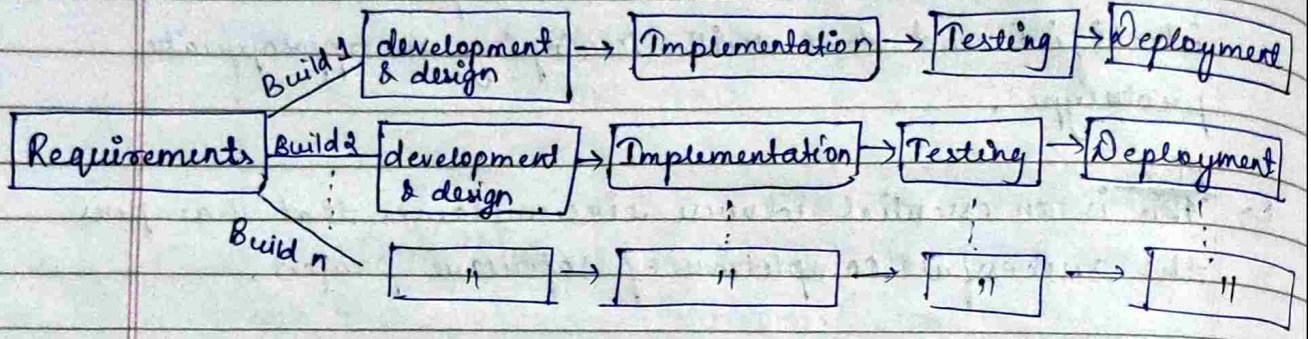
↳ The radius of spiral defines the cost of the project and the angular dimension represents progress made in current phase.

↳ Spiral model is known as meta model because it is the combination of waterfall, prototype & evolutionary model. For eg, the single loop spiral model represents the waterfall model. It uses a prototyping approach by building a prototype before the actual development of software.

↳ The iteration along the spiral can be considered as evolutionary levels through which the complete system is build.

↳ Hence, it enables to resolve risk at each evolutionary level along with using prototyping as risk reduction mechanism & also retains the systematic step wise method as waterfall model.

Iterative Model



↳ It starts with small set of software requirement & iteratively enhance it's version until the complete system is implemented or ready to deploy.

↳ Basic idea behind this model to develop a system through iterative cycle in a smaller portion of time.

↳ During software development, more than one iteration of software development cycle may be progressed at the same time, that's why it is called incremental build approach.

↳ Each iteration goes through requirement design to deployment phase and each subsequent release add functionalities of previous module.

18/08/25

RAPID Application Development

software development methodology that prioritize speed and flexibility through iteration prototypes and user feedback.

How RAD Works ? (Framework)

RAD ~~is structured~~ framework is structured around 4 key components that Guide entire development process.

(i) **Requirement gathering** :
Unlike traditional model, RAD doesn't need list of specifications. Instead stakeholders deliver high level requirements which will refine later.

(ii) **Rapid prototyping** :
After Requirement, development team focuses on working on prototype as quickly as possible.
goal : ⇒ construct refine application after users evaluation

(iii) **Construction** :
↳ After prototype acceptance, the fully functional system is constructed.
↳ Based on userfeedback and based on user requirement.

(iv) **Deployment** :
↳ After implementation, tested, refined it is deployed to live production environment.
↳ After that maintainance is carried out.

Phases in RAD Model Development

- **Buisness Modelling** — Info flow among buisness functions they quest/ans like who generates Ds what data drives, what data is generated ?
- **Data Modelling** — Accept data from Buisness model to refine the set of data, object that support development process.
- **Process Modeling** — processing description are created by adding, modifying & deletion data
- **Applicat^ Generation** — Automated tools are used to generate S/W.
- **Testing & Turnover** — Parts/Module must be tested then deployed Team n.

Team 1.

Agile Model

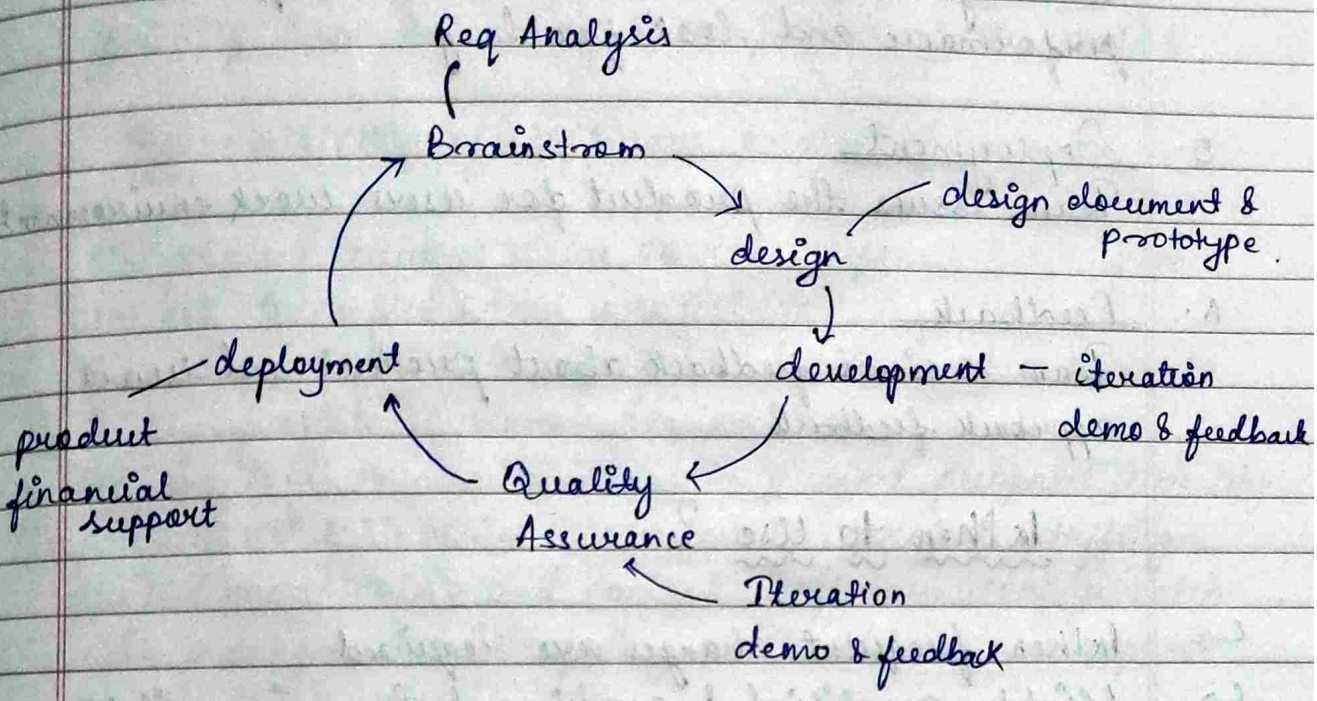
↳ Agile means swift (change quickly) & versatile (able to adopt many diff funct^)

↳ This approach is based on iterative development.

↳ This method breaks the tasks into small iterations or parts which does not require long term planning.

↳ Each iteration is considered as short time "frame" that typically lasts from one to four weeks.

↳ The division of entire project into smaller parts helps to minimize project risk and reduce project delivery time.



1. **Requirement gathering**
   - ↳ Define requirement
   - ↳ Explanation of time & efforts to build the project.
   - ↳ Also check feasibility in terms of economical & technical.

2. **Design the Requirement**
   - ↳ Using workflow diagram & high level UML to show the work of new features.
   - ↳ Also show how it will apply to existing system.

3. **Development**
   - ↳ Working on project get started to deploy.
   - ↳ Here, project goes various stages of improvement.

4. **Testing or Quality Assurance**
Quality Assurance team examines the product performance and look for bug.

5. **Deployment**
Team issues the product for users work environment.

6. **Feedback**
Team recieves feedback about product and work approach feedback.

When to Use ?

- When frequent changes are required.
- Highly qualified & experienced team is available.
- When project size is small.

Agile Testing Methods

- Scrum
- Crystal
- extreme programming (Xp)
- lean software development. etc

UNIT-2

19/08/25

Software Project Management

It enables the group of software engineers to work efficiently towards successful completion of the project.

Responsibilities of software project Manager

- s/w project manager takes overall responsibilities for successfully completion of project.
- These responsibilities range from building of a team to highly feasible customer presentation.
- He/she takes responsibilities like project proposal writing, project cost estimation, scheduling, project staffing, project monitoring and control, software configuration, risk management, interfacing with clients and report writing, presentation etc.
- These whole steps can be defined as three parts such as project planning, project monitoring & control activities.
- The project planning stage is done before developing the project and monitoring phase is done with ongoing project development.

Skills necessary for software project Manager

- It should have good qualitative judgement and decision-making capabilities.
- It should have past experience and good skills on cost-estimation, risk management, configuration management etc.
- It should have good communication skill for communicating with n-users.
- It should have sound knowledge of project management techniques.

Project Planning

↳ Once a project is found to be feasible, the software project manager undertake project planning which is done before the development activities.

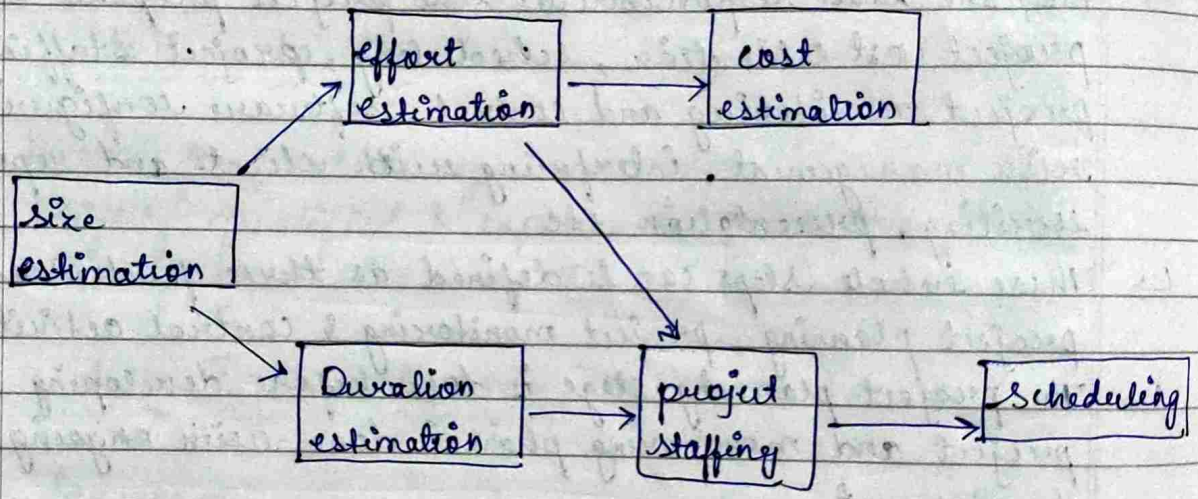
↳ These planning include several activities :

i) estimating the attributes like cost, duration & effort that required for a project.

ii) scheduling man power and resources.

iii) staff organisation and staffing plans.

iv) Risk identification and analysis planning.



[Procedence Ordering of project planning]

04/08/25

Date
Page

Monalitha

Metrices for Project size Estimation

Project size ⟶ Time with effort

- LOC (Lines of code)
- Function point ⟶ feature point

↳ In order to accurately estimate the project size, we need to define some appropriate metric.

↳ The problem size is defined as the content of software project whereas the project size is the measurement of complexity in terms of efforts & time required to develop a product.

↳ Currently there are 2 type of metrice such as
  (i) LOC
  (ii) Function point

LOC (Lines of code) :

↳ It is the simplest among all metrice to estimate project size.

↳ It is very popular as the project size is estimated by counting the no of source instruction in a developed program in which the comment lines and header lines are ignored.

↳ It holds some advantages along with many disadvantages.

Disadvantages :- i) LOC count at the beginning of project is very difficult.

ii) LOC gives a numerical value of problem size that can vary with individual coding style as we know different programmers have different coding style.

main disadv. iii) LOC focus ~~on~~ only on the coding activity & rarely computes the source line in a final program & ignore overall complexity & efforts to solve a problem.

iv) It Poorly co-relates quality and efficiency of the code.

v) It measures on lexical complexity rather than structural or logical complexity.

vi) It can only accurately calculated only after the code has been fully developed.

Function Point :

↳ It is developed by Albrecht in 1983.

↳ The most important advantage of function point is that it can be used to estimate the size of software product directly from problem specification part.

↳ For this metric, the size of software product can be directly dependent on the no. of different functions & features it supports.

↳ The formula of Function point is denoted as

$$FP = UFP * TCF$$

UFP = Unadjusted Function Point

TCF = Technical complexity factor

~~the UFP can be calculated~~

↳ It consider 5 functions of a system i.e, ~~no of inputs~~,

no. of input

no. of output

no. of inquiries (interactive queries)

no. of files (logically related data)

no. of interfaces (used to exchange info between system)

↳ So, UFP can be calculated as →

$$UFP = [(\text{no. of input} * \text{weight}) + (\text{no. of output} * \text{weight}) + (\text{no. of inquiry} * \text{weight}) + (\text{no. of files} * \text{weight}) + (\text{no. of interfaces} * \text{weight})]$$

Vector sum of 14 factors → DI

↳ TCF ~~defines~~ refines the UFP majors by considering 14 other factors. So, TCF can be calculated as :-

$$TCF = 0.65 + 0.01 * DI$$

DI = Degree of influence (0 to 70)

TCF can vary from 0.65 to 1.35

~~Feature point me~~ Feature Point Matric :

↳ Feature point metric is the extension of function point metric that incoporate extra parameter into algorithm complexity.

↳ It ensures the computed size has more complexity of functions & greater efforts required to develop. That's why it's size should be larger as compared to simple function.

01/09/25

Project Estimation Techniques

It includes the estimation of project size, effort, duration, cost and resource.

There are three types :-

i) Emperical
ii) Heuristic
iii) Analytical

Emperical Estimation Techniques

↳ In this, past experiences and educated guess play important role.

↳ It is useful in prior/early stage of software development.

↳ It is of two types :-

(a) Expert Judgement method
(b) Delphi cost method.

(a) Expert Judgement Method (informal, biased, unstructured)

↳ In this, an estimate is given by an experienced expert or a group of experts after analysing the project.

↳ In this, expert studies project modules or sub-systems and gives an estimation (cost or effort) based on past experience & knowledge.

↳ Sometimes, it can be content multiple experts.

↳ It is a quick and easy process but it includes bias values (due to personal opinion, over confidence & lack of knowledge in some area)

↳ Different experts may give different results.

(b) Delphi-cost Estimation

↳ It is a structured group estimation method to reduce bias in expert judgement.

↳ In this, a coordinator distribute the project detail (SRS document) to all the experts.

↳ Each expert independently give estimates without any group discussion.

↳ The coordinator collects responses and share a summary back with experts.

↳ Then experts review the summary and make changes or revise estimates.

↳ This process is repeated for several rounds until the ~~consensus~~ consensus reached.

↳ Then coordinator compiles the final estimates

↳ It reduces the individual bias, more reliable but it is a time consuming process.

02/09/25

Heuristic Technique

↳ It assume that the relationship among different project parameter can be model using suitable mathematical expression.

↳ Once the independent variable value is determined it will be easy to calculate the value of dependent variable based on mathematical regression expression.
The expression can be single variable and multi variable.

Single variable expression :

$$\text{The estimated parameter} = c_1 * e^{d_1}$$

where, c_1 & d_1 = const
& e = characteristics of software.

Multi variable cost estimation model can be represented as

$$\text{Estimated Resource} = c_1 * e_1^{d_1} + c_2 * e_2^{d_2} \ldots\ldots + c_n * e_n^{d_n}$$

$e_1, e_2 \ldots\ldots e_n$ = Independent variable
$c_1, c_2 \ldots\ldots c_n$ = constant
$d_1, d_2 \ldots\ldots d_n$ = constant

* The example of multivariable expression is Cocomo model.

Analytical Technique

↳ This technique derive the required result starting with certain basic assumption regarding the project.
↳ It is unlike emperical & heuristic technique because it has a scientific basics.
Eg : Halstead's software science.

GMP

COCOMO - A heuristical estimation technique

↳ COCOMO stands for Constructive Cost Estimation Model that was proposed by Boehm in the year 1981.

↳ Boehm postulated that any software development project can be classified into 3 categories.
i) Organic
ii) Semidetached
iii) Embedded
based on the characteristics of product & development ~~environment~~ environment.

↳ Organic, Semidetached & embedded were priorly known as application, utility and system program respectively.

↳ Brooks in 1975, states that utility programs are roughly three times difficult than application program and system program is again three times difficult than the utility program.

↳ So, he calculated the ratio of relative level of product development complexity for three categories (application, utility, system) is 1:3:9.

↳ The three categories of the system:

(i) Organic

↳ It deals with well understood application program where the development team is small and they are familiar to that kind of project.

(ii) Semidetached

↳ Here, the development team is the mixture of experienced & inexperienced staff.

↳ It deals with limited experience on limited system.

↳ It is refined version of organic system.

(iii) Embedded

↳ Here, the developed software is strong coupled with complex hardware.

↳ It is based on some regulations and operational procedures.

19/09/25

Basic COCOMO Model

↳ The basic COCOMO model gives an approximate estimation of project parameters.

↳ It is based on calculating two parameters i.e, efforts and Tdev (development Time)

↳ The equation is given as

$$\text{Efforts} = a_1 * (\text{KLOC})^{a_2}_{b_2} \quad \text{PM} \rightarrow \text{Person per month}$$

$$\text{Tdev} = b_1 * (\text{efforts})_{b_2} \quad \text{months}$$

where,

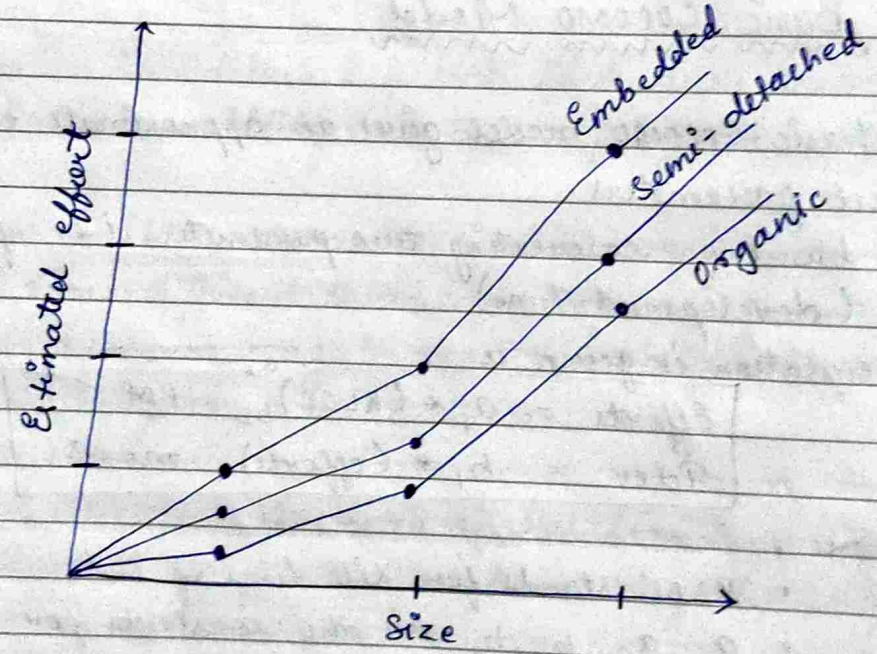
- KLOC stands for Kilo Lines of code
- a_1, a_2, b_1, b_2 are the constants for each category of software product.
- Tdev stands for estimated time to develop the s/w that is expressed in months
- Efforts is the total effort required to develop the s/w product that expressed in persons-months (PM)

Estimated development effort

Organic ⇒ efforts = $2.4 (KLOC)^{1.05}$ PM
Semidetach ⇒ efforts = $3.0 (KLOC)^{1.12}$ PM
Embedded ⇒ efforts = $3.6 (KLOC)^{1.20}$ PM

Estimated development time

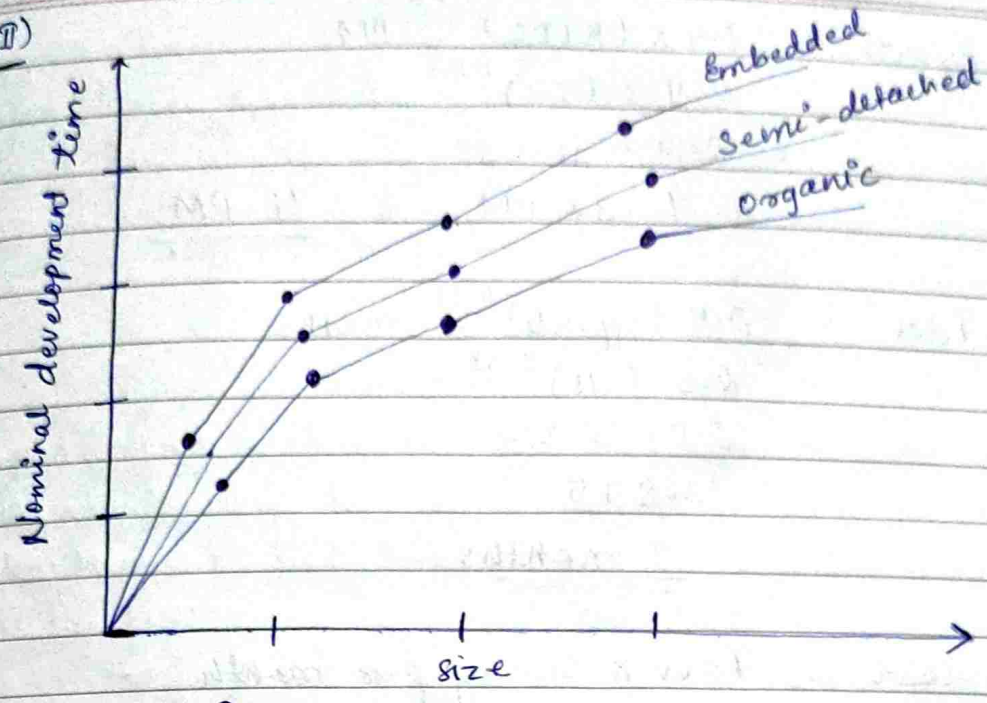
Organic ⇒ Tdev = $2.5 (efforts)^{0.38}$ months
Semidetach ⇒ Tdev = $2.5 (efforts)^{0.35}$ months
embedded ⇒ Tdev = $2.5 (efforts)^{0.32}$ months



Effort vs product size

Graph (I)

graph (II)



Development time vs size

↳ We can get some insight from the graph (I).
↳ Shows a plot between estimated effort and product size.
↳ We can observe that the effort is somewhat super linear with the size of the product.
↳ Hence, the efforts increases rapidly with the project size.

In graph (II)

↳ We can observe that the development time is sub-linear with the size of the product that means when the size of the product increases two times, the development time doesn't double but it increases moderately.
↳ This is because with the increasing of product size the large no. of activities are carried out simultaneously that reduces the time to complete the project.

Q⇒ Assume that the size of an organic type s/w product has been estimated to be 32,000 LOC, the avg salary of s/w engineers is 15,000 per month. Determine the efforts required to develop the s/w product & nominal development time.

Ans ⇒ Efforts = $2.4 \times (KLOC)^{1.05}$ PM

$= 2.4 \times (32)^{1.05}$

$= 2.4 \times 38.05$

$= 91.32$ PM $\approx$ 91 PM

Tdev = $2.5 (efforts)^{0.38}$ months

$= 2.5 \times (91)^{0.38}$

$= 2.5 \times 5.55$

$= 13.875$

$=$ 14 months

cost = Tdev x Salary per month

$=$ 14 x 15000

$=$ 110000

15/09/25

Intermediate COCOMO Model

↳ The Basic COCOMO model assume that the efforts and development time are the functions of product size along.

↳ However, beside these parameters other project parameter can affect the development of the product.

↳ So, the intermediate COCOMO model recognises the fact and refines initial estimate obtained through basic COCOMO model.

↳ It can be defined as the basic COCOMO model with 15 cost drivers (multipliers).

↳ This cost drivers is divided into 4 classes :-

i) Product

ii) computer

iii) personnel

iv) development environment

i) Product :
The characteristics of the product are considered include inherent complexity, reliability and requirements of the product.

ii) Computer :
The characteristics of computer that are considered includes the execution speed, storage space etc.

iii) Personnel :
The attribute of development personnel that includes experience, programming capacity, analysis capacity etc.

iv) Development Environment :
↳ Development environment attributes captures the development facilities available to the developers.
↳ An important parameter that is considered is sophistication of automation tool (CASE - Computer Aided Software Engineering) for software development.

Complete COCOMO Model

↳ A major shortcomings of both basic and intermediate COCOMO model that they consider a software product as a single homogeneous entity.

↳ The complete COCOMO model consider these differences in the characteristics of sub-system and estimate the effort and development time as the summation of these sub-systems.

↳ Let us consider the application of complete COCOMO model i.e. MIS system (a distributed management system) which may have the following components :-

(i) Database part
(ii) GUI part
(iii) Communication part

↳ So, these are different subsystems. According to complete COCOMO model, the cost of all these three components can be estimated separately & sumed up to give the overall cost of the system

Halstead's Software Science/Method - An Analytical Method

↳ Halstead technique measures the size, development efforts and development cost of the software product.

↳ For a given program,

n_1 = no. of unique operators
n_2 = no. of unique operands
N_1 = Total no. of ~~operands~~ operators
N_2 = Total no. of operands

Calculation of length and vocabulary

Length = total no. of operators & operands

$$N = N_1 + N_2$$

Vocabulary = Total no. of unique operators & operands

$$n = n_1 + n_2$$

Program volume is denoted as V i.e, minimum no. of bits needed to encode the program.

$$V = N \log_2 n$$

Potential Minimum volume is denoted as V^* i.e, defined as the volume of most unique program in which the problem can be coded.

Suppose, the data are $d_1, d_2 \ldots d_n$ where $n_1 = 2$, $n_2 = n$ the unique program would be $f(d_1, d_2 \ldots d_n)$

$$\boxed{V^* = (2 + n_2) \log (2 + n_2)}$$

The program label $\boxed{L = V^*/V}$

Effort $(E) = L/V = V^*/V/V \Rightarrow \boxed{E = V^*/V^2}$

Estimated time $\boxed{T = E/S}$. S = Speed of mental discrimination

Estimated program length is given by

$$\boxed{N^\wedge = n_1 \log n_1 + n_2 \log n_2}$$

16/09/25

Staffing Level Estimation

↳ After requirement analysis done, it is necessary to determine the starting requirement of the project.

↳ Norden first investigated the starting pattern of research and development project.

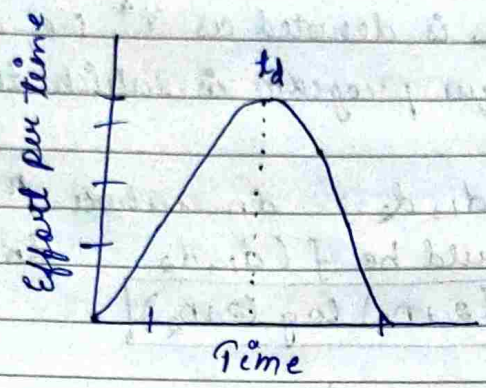
Norden's Work

He found that starting pattern can be approximated using Rayleigh distribution curve and this curve is given by the equation $\boxed{E = \frac{K}{t_d^2} * t * e^{-t^2/2t_d^2}}$

where E = effort required at the time t

K defines the area under curve

t_d defines the curve at which it attains max value.



Putnam's work

↳ He studied the problem of staffing of software project and found that the software development has similar characteristics studied by Norden.

↳ By analysing large project, Putnam provided the expression i.e, $\boxed{L = C_K K^{1/3} t_d^{4/3}}$

where, L = product size in KLOC
K = total efforts (PM)
t_d = time of system integration & testing
C_K = state of technology function

$$K^{1/3} = \frac{L}{C_K t_d^{4/3}} \qquad \left(C = L^3/C_K^3\right)$$

$$\Rightarrow (K)^{3/3} = \frac{L^3}{C_K^3 t_d^4}$$

$$\Rightarrow K = \frac{L^3}{C_K^3 t_d^4} \Rightarrow \boxed{K = \frac{C}{t_d^4}}$$

$\because K \propto \frac{1}{t_d^4}$

so, $\boxed{\frac{K_1}{K_2} = \frac{t_{2d}^4}{t_{1d}^4}}$

↳ The putnam's estimation model works well for large systems but seriously over-estimates the efforts in medium or small system.

Jensen's work / Model (1984)

↳ It is similar to Putnam's model but it is applicable to smaller and medium size model in which the expression is given as $\boxed{L = C_{te}\, t_d\, K^{1/2}}$

where, C_{te} = technology constant
t_d = development time
K = Efforts

↳ By analysing the efforts equation, we can define

$$\boxed{\frac{K_1}{K_2} = \frac{t_{d_2}^2}{t_{d_1}^2}}$$

Scheduling

↳ It is an important project planning activity that involves deciding which task would be taken when.

↳ It follows the no of list :-

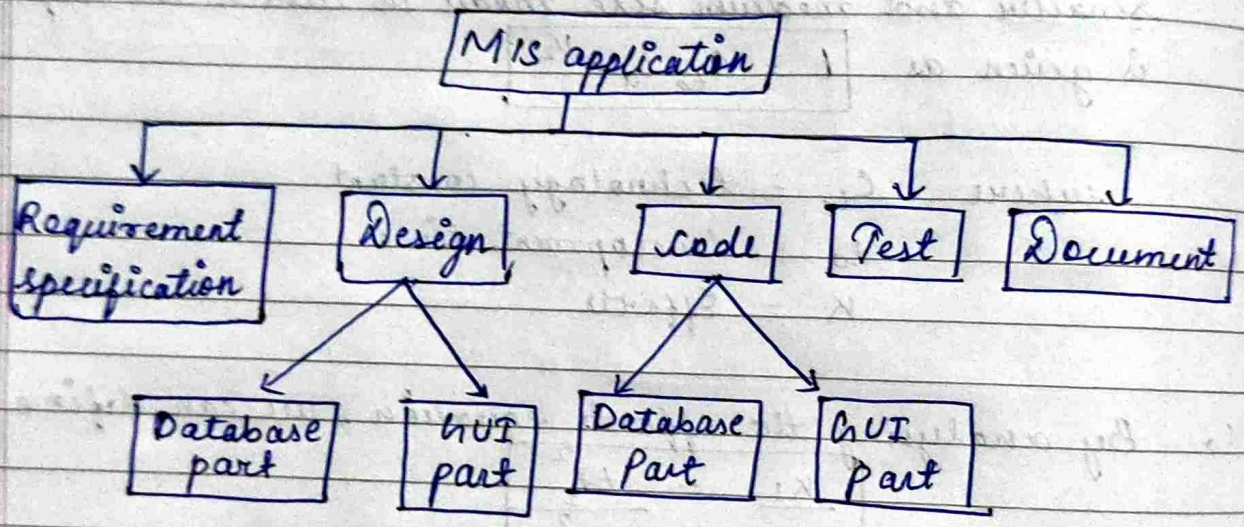
i) Identify all task needed to complete the project.
ii) Breakdown the large task into smaller activities.
iii) Determine the dependency among different activities.
iv) Establish the necessary estimates to complete the activities
v) Allocate resource to activities.
vi) Planning the starting and ending day.
vii) Determine the critical path (it determines duration of the project)

Work Breakdown Structure (WBS)

↳ WBS is used to decompose a given task set recursively into small activities.

↳ WBS provides a notation for representating the major tasks needed to be carried out in order to solve a problem.

↳ Here is a WBS of a MIS problem.



↳ While breaking down a ~~task~~ larger task into smaller subtasks, the manager has to make some hard decisions.

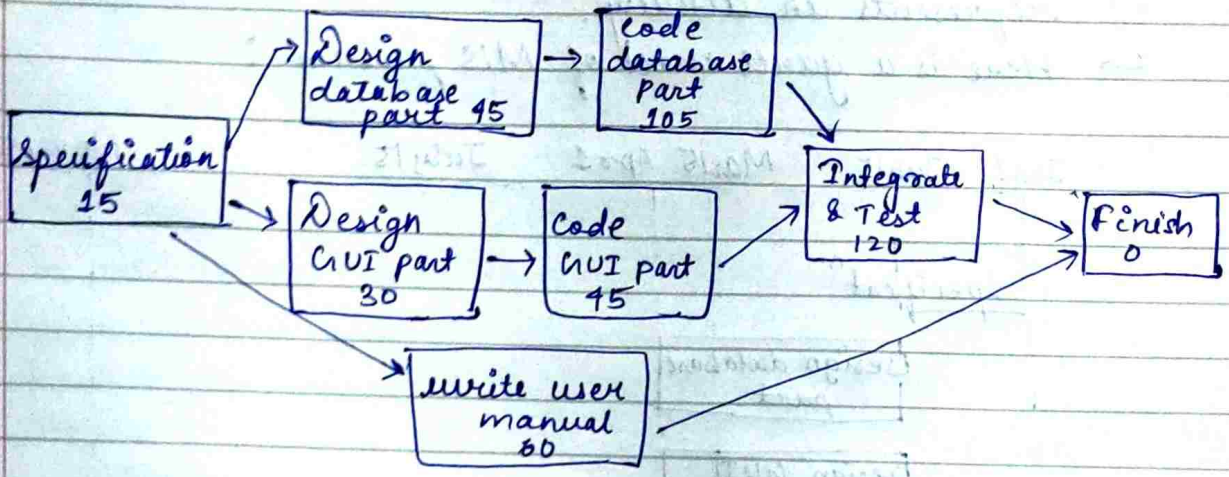
↳ If the task is broken down into a large no. of very small activities, these can be distributed to a large no. of engineers.

↳ If the activity ordering permits, the solutions to these can be carried out independently by which it becomes possible to develop the product faster.

Activity Networks and Critical Path Method

↳ An activity network shows the different activities making up a project, their estimated durations & interdependencies.

↳ Here is a activity network representation of the MIS problem :



↳ Managers can estimate the time durations for the different tasks in several ways :

(i) empirically assign durations to different task → not a good idea

(ii) let engineer estimate the time for an activity assigned

↳ A good way to achieve accuracy in estimation of the task duration without creating undue schedule pressures is to to have people set their own schedules.

Critical Path Method

↳ It help to determine both longest and ~~for~~ shortest possible time taken to complete a project.

↳ There are three ~~elem~~ essential elements :

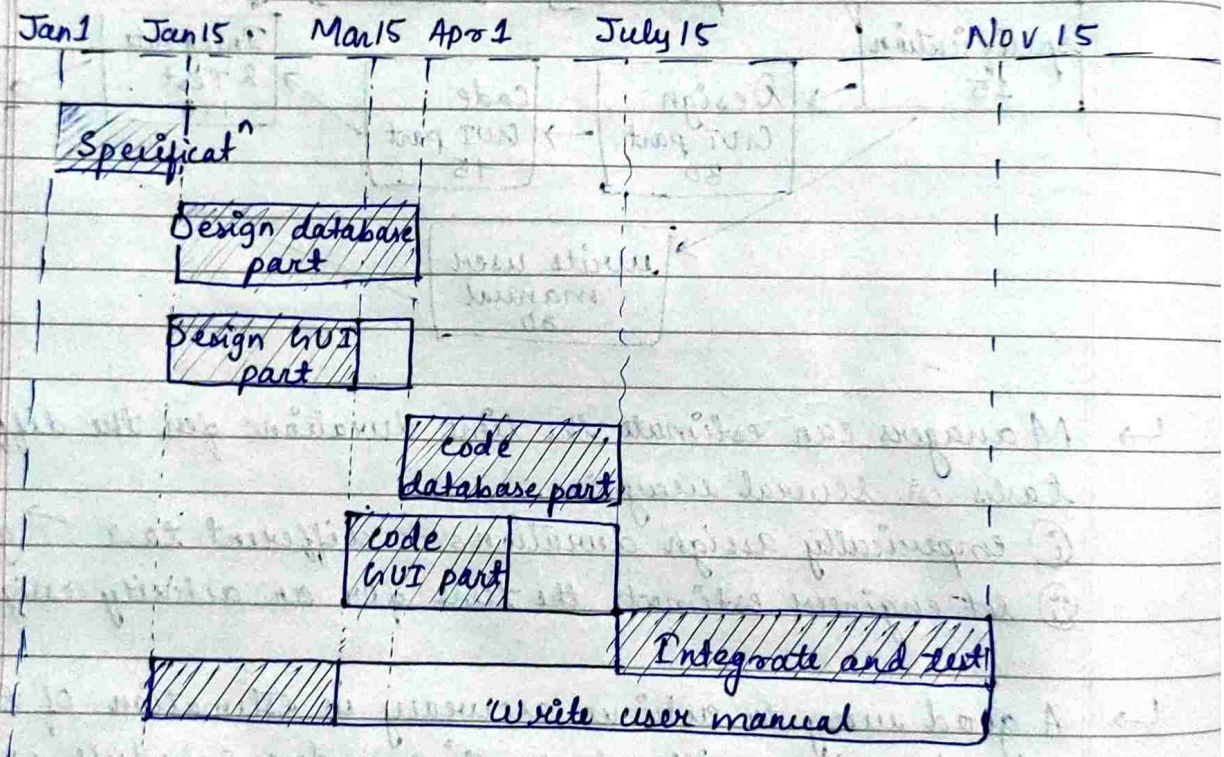
(i) the task required to complete the project.

(ii) which task depends on completion of others

(iii) A time estimate for each activity

Gantt Charts

↳ Gantt charts are mainly used to allocate resources to activities.
↳ The resources allocated to activities include staff, hardware and software.
↳ Gantt charts are very useful for resource planning.
↳ They are special type of bar chart where each bar represents an activity.
↳ Here is a gantt chart of MIS problem :



The shaded part ⇒ length of time each task is estimated to take.
The white part ⇒ Slack time i.e, the latest time by which a task must be finished.

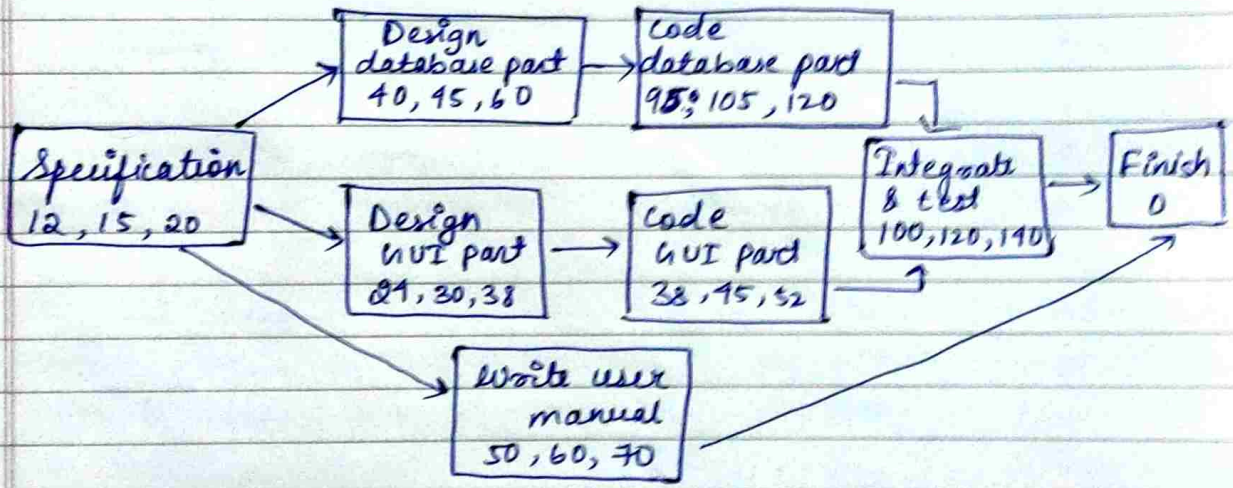
PERT Charts (Project Evaluation & Review Technique)

↳ PERT chart represents the statical ~~version~~ variations in the project estimates assuming a normal distribution.

↳ In PERT chart, instead of making single estimate for each task, pessimistic, likely and optimistic estimates are also made.

pessimistic = unfavourable + risk
likely = unfavourable + ~~favourable~~ favourable
optimistic = only favourable (no risk)

↳ PERT charts are more sophisticated forem of activity chart. In activity chart, only estimated task durations are represented and since the actual durations might vary from the estimated durations, the utility of activity diagram is limited.



PERT chart of MIS problem.

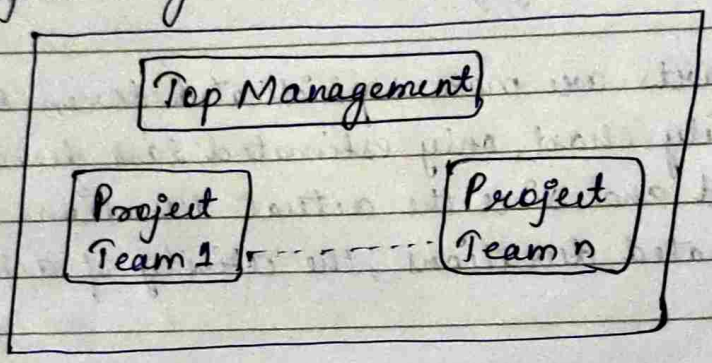
18/09/25

Organisation and team Structure

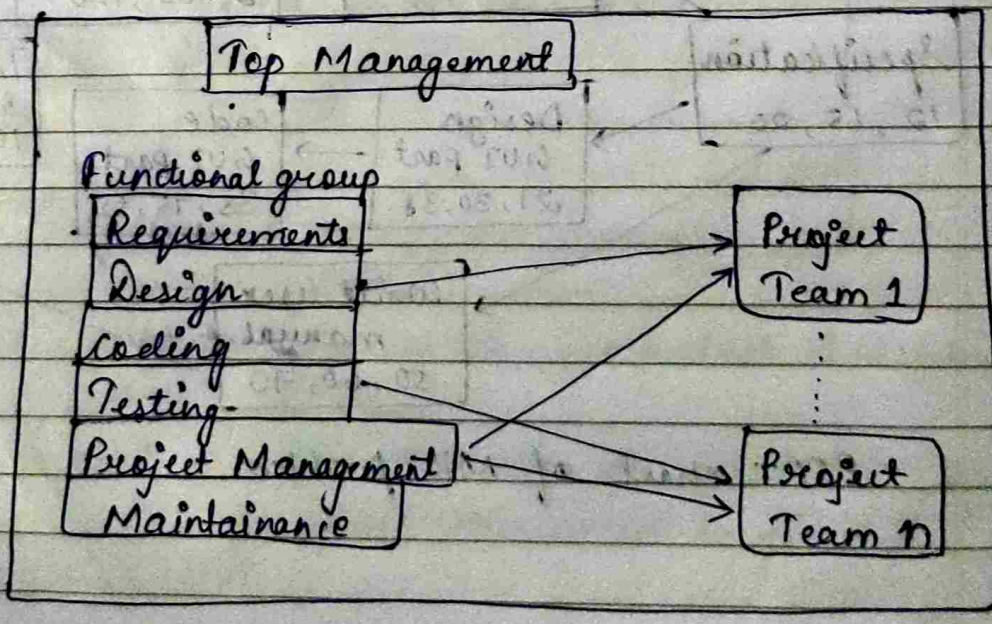
↳ In software organisation, there assigned different teams of engineers to handle different projects.

↳ There are essentially two broad ways in which a software development organisation can be structured. that are functional format and project format.

(a) Project Organisation



(b) Format Organisation



↳ In the project format (a), a set of engineers are assigned to the project at the start of the project and remain till the completion of it.

↳ In functional format (b), different teams of programmers perform different phases of a project.

↳ For example, 1 team may do requirement specification and other may design coding part.

↳ Here, the partially completed product passes from one team to another as the product evolves.

↳ Here, communication plays important role for better understanding of working of project and it requires good quality documentation after every activity.

Advantages :—

i) Easy staffing
ii) Production of good quality documents
iii) Job specialisation
iv) Efficient handling the problem.

Team Structure

↳ Team Structure addresses the issue of organisation of individual project teams.

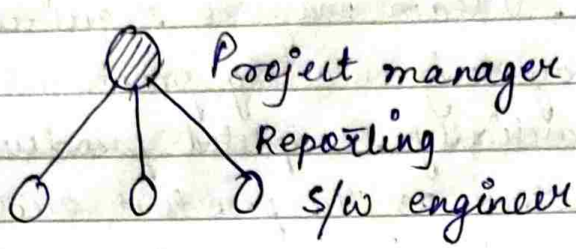
↳ There are three format in ~~project~~ team structure like democratic, chief programmer and mixed team organisation

Chief Programmer Team :

↳ It is the formal structure in which a senior engineer provides the technical leadership and designated as chief programmer.

↳ He partioned the task into small activities, assigned the team members and verifies with integrates the product developed by engineers.

↳ It is more efficient and provides better understanding of the project.



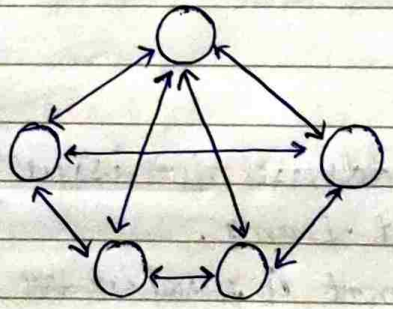
Democratic :

↳ As the name implies, there is no formal team hierarchy which provides higher job satisfaction.

↳ In this, the project members can communicate independently with each other.

↳ It is appropriate for less understood problems and suitable for research oriented problems.

↳ For large size project, it can be chaotic.



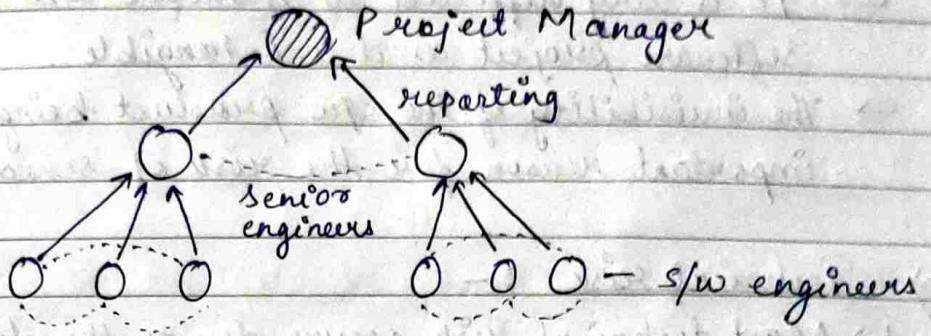
O = S/w engineers
↔ = communication path

Mixed Organisation Structure :

↳ It is the combination of formal and informal structure that incorporates both heirarchial reporting and democratic set-up.

↳ It is suitable for large team size and complex programs.

↳ The communication between software engineers follows the democratic structure and the communication between senior engineer and project manager follows the chief programmer structure.



~~23/09/25~~

Risk Management

↳ A risk is an essential adverse circumstance which hampers the successful completion of project development.

↳ ~~If a risk becomes true,~~

↳ It is necessary to anticipate and identify different risks that a project may be susceptible to.

↳ Risk Management consist of three essential activities:
- (i) risk identification
- (ii) risk assessment
- (iii) risk containment.

(i) Risk Identification :

↳ The project manager needs to anticipate the risks in the project as early as possible so that the impact of the risk can be minimized by making effective risk management plans.

↳ There are three main categories of risks which can affect a software project :

a) Project risk :-
↳ An important project risk is schedule slippage.
↳ It is very difficult to monitor and control a software project as it is intangible.
↳ The invisibility of the s/w product being developed is an important reason for the risk of schedule slippage.

b) Technical risk :-
↳ Most technical risk occurs due to the development team's insufficient knowledge about the product.
↳ Technical risk includes ambigious specification, incomplete specification, changing specification, technical uncertainity and technical obsolence.

c) Buisness risk :-
↳ Buisness risk include risks of building an excellent product that no one wants, losing budget or personal commitments etc.

(ii) Risk Assessment :

↳ The objective is to rank the risks in terms of their damage causing potential.
↳ Each risk should first be rated in two ways :-
a) The likelihood of a risk coming true (r)
b) The consequence of a problem associated with that risk (s).

Based on these two factors, the priority of each risk can be computed as

$$p = r * s$$

p → priority with which risk must be handled
r → probability of the risk becoming true
s → severity of damage caused due to risk becoming true.

(iii) **Risk Containment :**

↳ Different risk require different containment procedures after the risk is identified and are assessed.

↳ There are three main strategies for risk containment :

a) **avoid the risk :-**
This may take several forms such as discussion with the customer to reduce the scope of work and give incentives to engineers to avoid risk of manpower turnover

b) **Transfer the risk :-**
This strategy involves getting the risky component developed by a third party.

c) **Risk reduction :-**
This involves planning ways to contain the damage due to a risk.

Risk Leverage ⇒ It is the difference in risk exposure divided by the cost of reducing the risk.

$$\text{Risk leverage} = \frac{\text{risk exposure before reduct}^n - \text{risk exp after reduct}^n}{\text{cost of reduct}^n}$$

05/09/25

Software Configuration Management

↳ The result of large software development effort typically consist of large no of objects i.e, source code, design document, SRS document, test document etc.

↳ The states of all these objects at any point of time is called configuration of software product.

↳ For a reliable software development, these are must be changed.

↳ So, software configuration management deals with effectively tracking and controlling the configuration of software product during it's lifecycle.

Necessity :

↳ Mainly software configuration management is to control access to different deliverable objects.

↳ If it is not done correctly then following problems will be arise :

i) inconsistancy when the objects are replicated.
ii) problems associated with concurrent access
iii) fails to provide stable development environment
iv) System accounting and maintaining status information

Configuration management activities

Configurat" Identificat"
deciding which part of system should be kept track of

Configuration control
ensures that the changes to system happens smoothly.

Configuration Identification :

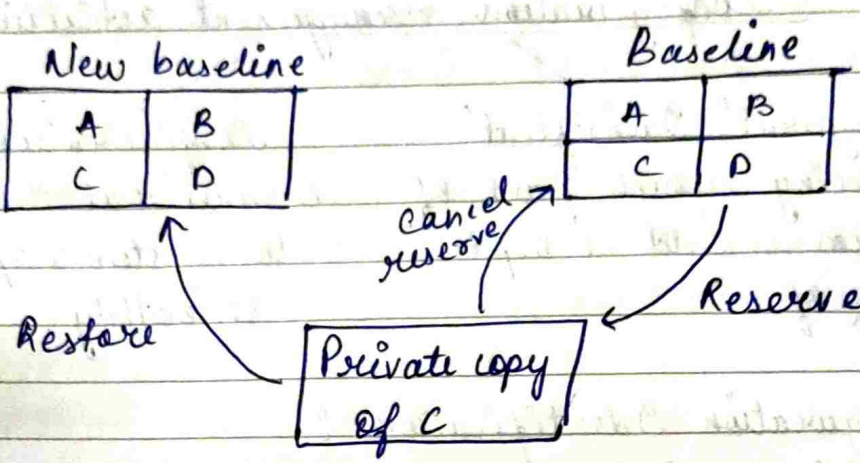
↳ For this, project manager classifies the object as
a) controlled (already under configurat" control)
b) pre-controlled (are not yet controlled will eventually)
c) uncontrolled (will not subject to configurat" control)

Controllable Objects :

↳ Requirement specification document.
↳ design
↳ Tools, source code etc.

Configuration Control

↳ process of managing changes to controlled object.
↳ prevent unauthorised changes to any controlled object.
↳ In order to change controlled object such as module a developer can get a private copy of a module by reserve operation that means its allow only one person to reserve module at any time at reserve time, no other can reserve it. Only after it restored other can interfare.
That solves the problem associated with concurrent acces.



Source code control system (SCCS)

26/09/25

UNIT - 3

Requirement Analysis and Specification

↳ The requirement analysis & specification phase starts once the feasibility study is completed and the project is found to be financially and technically feasible

↳ This phase consist of two activities such as :-
  i) Requirement gathering & analysis
  ii) Requirement specification

i) Requirement gathering and analysis

↳ The analyst starts the requirement gathering and analysis activity by collecting all informations from the customer.
↳ In pratice, it is very difficult to gather necessary information and to form unambigious understanding of a problem.
↳ As the collected requirements are noisy in nature, so it should be extracted.

(a) Requirement gathering :-

↳ The activity typically involves interviewing end users and study existing document to collect ~~to~~ all possible information regarding the system.

(b) Analysis of gathered requirements :-

↳ The main purpose of this activity is to clearly understand the exact requirements of the customer.
↳ Here, the analyst has understood the exact customer requirement and proceed to identify and resolve various requirement problems.
↳ These problems are : (1) Anamoly
  (2) Inconsistency
  (3) Incompleteness

(1) Anamoly → It is an ambiguity in requirements that means there are several interpretion of specific requirement. so, analyst has to eliminate these anamolous requirements.

(2) Inconsistancy → Analyst eliminates the requirements which are inconsistance that means any one of the requirement contradict another.

(3) Incompleteness → It is the one where some parts of the requirement are omitted. It is caused by inability of customers to visualize and anticipate all the features.

14/10/25

ii) Software Requirement Specification (SRS)

↳ After the analyst first collected all the required information regarding the software to be developed that is free from incompleteness, inconsistencies and anamolies.

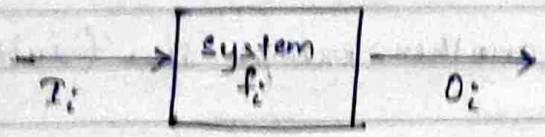
↳ After that, the requirements are systematically organised in a form i.e, SRS document.
The SRS document is the tricky job in s/w development project.

Contents of SRS document

There are mainly three contents in SRS document.

(i) Functional requirement
(ii) Non-functional requirement
(iii) Goal of implementation

(i) The functional requirement of the system are documented in SRS document should clearly describe each function which the system should support with corresponding input and output dataset.



(View of a system that performing set of functions)

(ii) Let us consider a system performing a set of functions f_i. Each function (f_i) can be considered as the transformation of set of Input data I_i to corresponding set of output data O_i.

(ii) The non-functional requirement deals with the characteristics of the system that can't be express as functions.
Non-functional requirement includes aspect concerning, maintainibility, portability, usability, reliability and accuracy of results.

(iii) The goal of implementation part give some general suggestions regarding development. These suggestion guide trade-off among design and decission.

Functional Requirements :

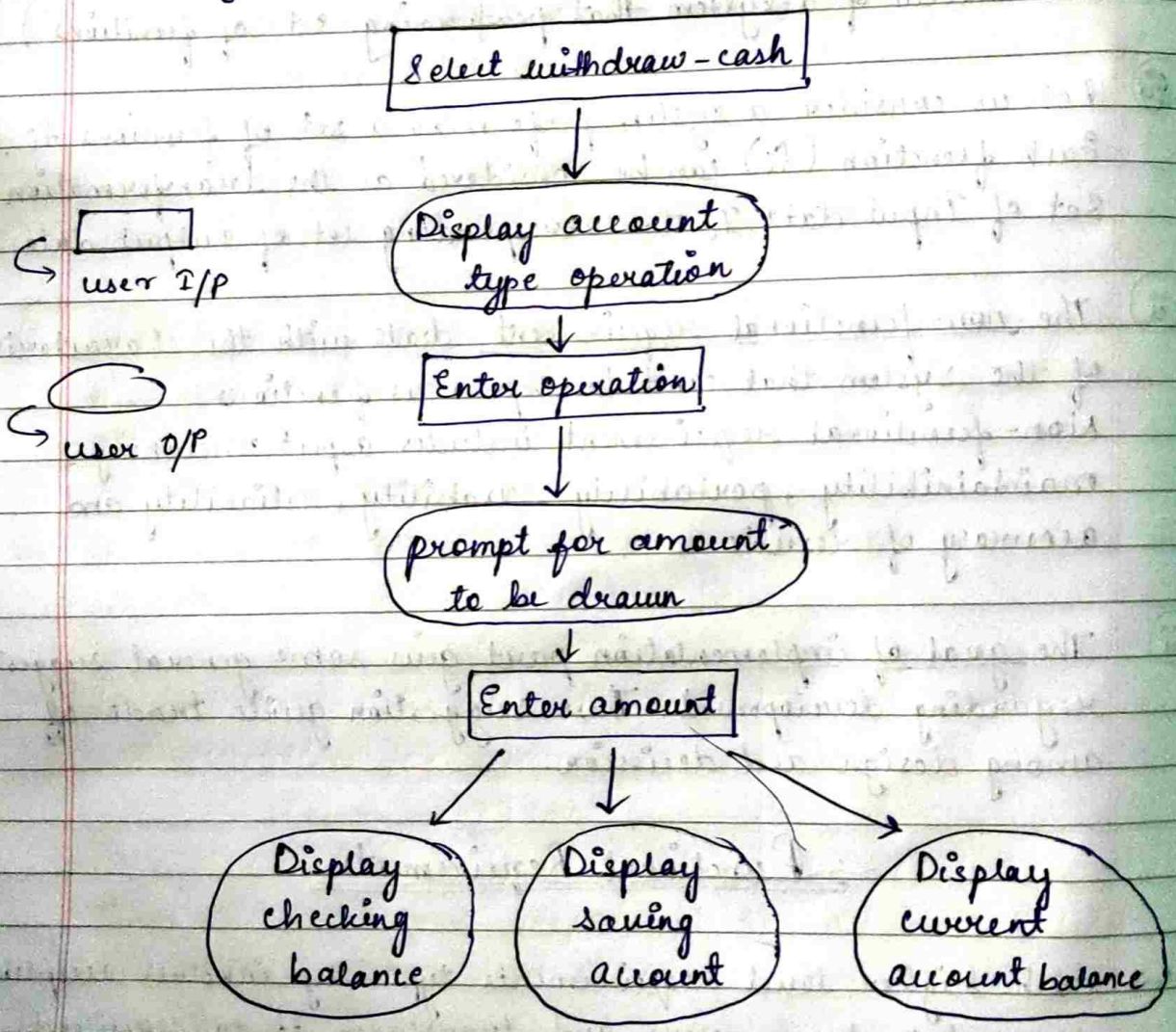
↳ Each higher level requirements typically involves accepting some data from users and transform it to required response to users.

↳ For eg - In library automation software, a high level functional requirement might be a "search book".

↳ These function ensure accepting the book name or set of keywords from the users and learning a matching algorithm and provides the matched entity to the user

↳ Hence, high level functional requirement may involve number of interaction between system and multiple users

Let us consider another example like (withdraw cash) in ATM system.



[Interaction between users and system in withdraw cash high level function requirement]

Characteristics of Good SRS :

↳ It should be well-structured in nature (~~symm~~ systematically organize that helps to understand and modify it easily).

↳ Concise (it should be unambiguous, consistent and complete).

↳ Black Box view (only specify what the system should do rather than how to do)

↳ Conceptual Integrity (The readers can easily understand the content)

↳ Response to undesire events (It should accept responses to exceptional conditions)

↳ Verifiable (It determine whether or not requirements have been made in implementation).

↳ Traceability (The design components should be traced to specific requirements & vice versa).

Characteristics of BAD SRS :

↳ Overspecification :- ~~it aspect~~ it specify how to do aspect in SRS document.

↳ Forward References :- We should not refer to aspect that are discussed much later.

↳ Wishful thinking :- These type of problem concern description of aspect which is difficult to implement.

17/10/25

Organisation of SRS document

↳ SRS document is organised as below :-

(1) Introduction
   a) Background
   b) Overall description
   c) Environmental characteristics :
      (i) Hardware
      (ii) Peripherals
      (iii) people

(2) Goals of implementation
(3) Functional Requirements
(4) Non-functional Requirements
(5) Behavioural description
   a) System states
   b) Events and actions

↳ It includes functional and non-functional requirements and guidelines for system implementation

↳ The introduction section describe the context in which the system is being developed. An overall description of system and environmental characteristics describe the properties of environment in which the system will interact.

↳ Goal of implementation includes suggestions for developing the system.

↳ After the goal of implementation, the SRS document includes functional and non-functional requirements.

↳ The behaviour of the system can be specified using either finite state Machine (FSM) and any other formalism.

↳ FSM can be used to specify possible ~~ways~~ state of system and transition among the state due to occurance of events.

Techniques for Representing Complex logic

↳ A good SRS document should properly characterise the conditions under which different scenarios of interaction occurs which are sometimes complex.

↳ In case of complex condition, it is very difficult to analyze and comprehend / understand.

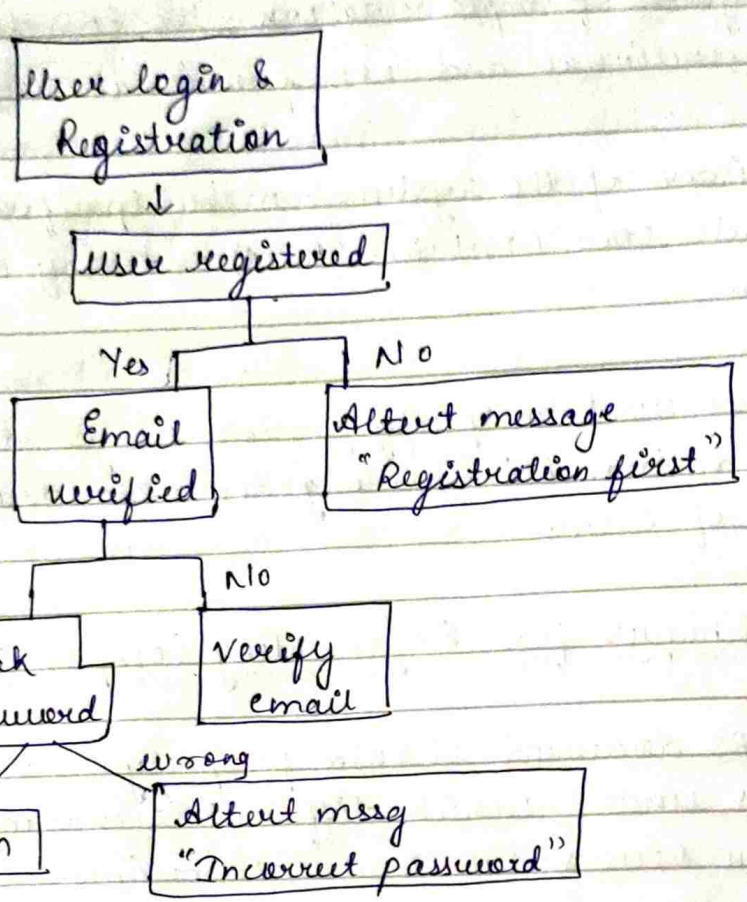
↳ In this condition, two main techniques are used to analyze and represent complex processing logic :
  a) decision tree
  b) decision table

a) Decision Tree

↳ It is the method of showing relationship between conditions and it's actions.

↳ It is a non-linear tree like structure which uses branching method in order to display all possible outcomes of any decision.

↳ Edge ⟶ decision
leaf node ⟶ action



b) **Decision Table**

↳ It is the tabular representation of all conditions & actions

↳ Components of decision table :-

condition stub	condition entry
Action stub	Action entry.

} 4 quadrants.

Condition stub ⇒ all conditions are checked and listed here.
Action stub ⇒ all actions are listed here.
condition entry ⇒ provide answers to questions.
Action entry ⇒ provide appropriate action from answers to conditions.

↳ User login and registration :-

Condition	Rule 1	Rule 2	Rule 3	Rule 4
User name	F	T	F	T
Password	F	F	T	T
Action	—	—	—	—
Output	Error	Error	Error	login

Formal System Development or Formal Specification Techniques :-

Formal Techniques :

↳ Mathematical method used to specify h/w & s/w system.
↳ Verify whether a specification is realizable (~~are~~ happened are to our need), whether an implementation satisfies it's specification or not.
↳ It consists of 2 sets 'syn' and 'sem'
1 relation between them 'sat'

syn = syntatic domain
sem = semantic domain
sat = satisfaction relation

Syntactic domain

↳ Consist of alphabets or symbols and set of formation rules to construct well formed formulas from alphabet.
↳ well formed formulas used to specify the system.

Semantic domain

↳ Abstract datatypes specification language are used to specify algebras, theories and programs.

↳ programming languages are used to specify functions from I/P to O/P values.

↳ Concurrent and distributed system specification language used to specify state sequences, event sequences, state-transition sequence, state machine etc.

Satisfaction Relation

↳ determined by using a homomorphism known as "semantic abstraction function" that maps the elements of semantic domain to equiclass.